

1. Bevezetés

1.1 Köszönetnyilvánítás

Köszönjük **Kardos Henriettának, Kárpáti Zoltánnak és His Imre Sándornak**, hogy az elmúlt években hiánytalanul tanítottak minket, és segítettek fejlődni a programozásban. Hálásak vagyunk azért a sok tudásért, türelemért és támogatásért, amelyet tőlük kaptunk. Külön szeretnénk megköszönni **Kardos Henriettának és His Imre Sándornak**, hogy végig segítették a projektünk fejlesztését, és tanácsaikkal hozzájárultak a munkánk elkészítéséhez.

1.2 Témaválasztás

A **MuscleMind Gym Tracker** témáját azért választottuk, mert mindketten járunk konditerembe, ezért ez a téma elég közel áll hozzánk. Edzés közben mi is tapasztaltuk, hogy sokszor nem elég csak lemenni edzeni, hanem jó lenne látni azt is, **hogy miben fejlődtünk az előző alkalmakhoz képest**. Fontosnak tartjuk, hogy a felhasználó vissza tudja nézni az **edzéseit**, a **gyakorlatait** és azt is, hogy **melyik izomcsoportot mikor edzette**. Emellett a táplálkozásra is szerettünk volna nagyobb figyelmet fordítani, mert szerintünk az edzés önmagában nem mindig elég a jó eredményhez.

Az ötlet onnan jött, hogy eddig nem igazán találtunk olyan alkalmazást, amely normálisan egyben kezelné az **edzések naplózását** és a **receptek ajánlását**. Vannak külön edzős appok, és vannak külön étkezéssel kapcsolatos appok, de mi egy olyan megoldást szerettünk volna készíteni, ahol ezek **egy helyen megtalálhatók**. A regisztráció után ezért egy **rövid kérdőív** segít abban, hogy az alkalmazás jobban igazodjon a felhasználóhoz. Ez alapján a profil személyesebb lesz, és az app nem teljesen általános módon működik.

Az alkalmazásba szerettünk volna olyan részeket is beletenni, amelyek tényleg segítenek abban, hogy a felhasználó átlássa a saját haladását. Ezért készült el a **statisztika oldal**, ahol vissza lehet nézni a mentett adatokat **1, 7 vagy akár 30 napos bontásban** is. Így nem csak érzésre lehet eldönteni, hogy történt-e fejlődés, hanem az **adatok alapján is látható**. Az **emberfigura** ötlete is emiatt került bele, mert sokkal könnyebb úgy értelmezni egy

edzéstervet, ha rögtön látszik, hogy az adott napon **melyik izomcsoportra fog edzeni a felhasználó**. Emellett fontosnak tartottuk, hogy legyen egy egyszerű kapcsolatfelvételi lehetőség is, ezért került az alkalmazásba a **ticket alapú kontakt rendszer**. Összességében egy olyan appot szerettünk volna készíteni, amit mi is hasznosnak tartanánk, és amit tényleg el tudnánk képzelni edzés mellett használni.

1.3 Használt technológiák

Node.js

- A webalkalmazás szerveroldali működéséhez használt technológia.
- A szerveroldali logika, az API-végpontok kezelése, a session-kezelés, az adatbázis-műveletek indítása és a felhasználói műveletek feldolgozása ezen keresztül történik.

HTML

- Az oldalak alapvető felépítéséhez használtuk.
- Ezzel alakítottuk ki a webapp szerkezetét és tartalmi részeit.

CSS / Bootstrap

- A weboldal megjelenésének formázására szolgált.
- Segítségével állítottuk be a színeket, elrendezéseket, méreteket és egyéb vizuális elemeket.

JavaScript

- A webalkalmazás működésének és interaktív részeinek elkészítéséhez használtuk.
- Kezeli például az API-kérések indítását, az űrlapok adatainak feldolgozását, a statisztikai adatok megjelenítését, a grafikonok betöltését és a felhasználói felület dinamikus frissítését.

MySQL

- Az alkalmazás adatbázis-kezelő rendszere.

- Ebben tároljuk többek között a felhasználói fiókokat, profiladatokat, edzésterveket, naptári edzésnaplókat, recepteket, allergéneket, statisztikákat, XP-adatokat, barátkapcsolatokat és támogatási jegyeket.

2. Fejlesztői dokumentáció

2.1. Projekt áttekintése

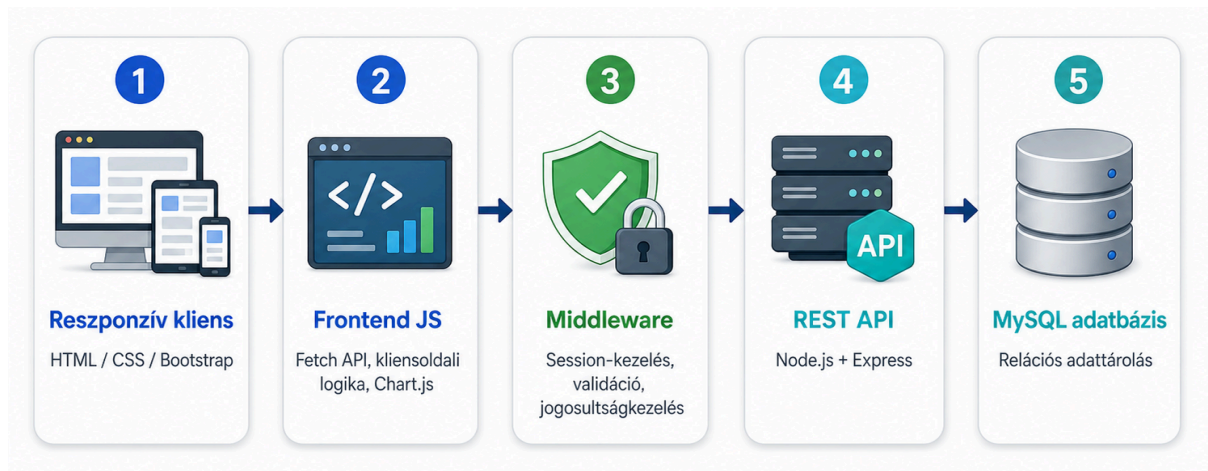
A **MuscleMind Gym Tracker** egy edzőközpontú **full-stack webalkalmazás**, amelyben a felhasználó **edzéstervet kezelhet**, **naptár alapján naplózhatja az edzéseit**, **követheti a statisztikáit**, valamint étrendjéhez és céljaihoz illeszkedő **recepteket kereshet**. A fejlesztői dokumentáció ezen része a projekt működésének, felépítésének és **technikai háttérének áttekintését** mutatja be, valamint a fontosabb algoritmusok tesztelését is magában foglalja.

A webalkalmazás nemcsak alapvető **profil- és edzésterv-kezelést** biztosít, hanem személyre szabott profiladatok alapján támogatja a felhasználót céljai elérésében és fejlődésének hosszú távú nyomon követésében is. A projekt kialakítása során fontos szempont volt, hogy a **felhasználói felület könnyen használható legyen**, miközben a háttérben jól strukturált **validációs folyamatok, jogosultságkezelés, adatkezelés és elkülönített backend-logika** biztosítja a webalkalmazás gördülékeny és stabil működését.

A projekt struktúrája:

- **backend/**: Node.js / Express alkalmazás
- **frontend/**: statikus HTML/CSS/JavaScript kliensoldal
- **backend/sql/**: MySQL adatbázis-séma és kapcsolódó adatbázis-kezelés
- **backend/middleware/**: kérések validációja és jogosultságkezelése
- **backend/api/**: REST API végpontok

2.2. Architektúra



2.1. ábra: A MuscleMind rendszer architektúrája

2.2.1 Backend

A backend szerver **Express 5**-re épül. A belépési pontot a **backend/server.js** fájl jelenti, ami kezeli a statikus frontend fájlok kiszolgálását, a session-kezelést, a routingot, valamint a központi hibakezelést is.

Az **API** végpontok a **backend/api/** mappában találhatók, külön modulokra bontva. Ez a felépítés teszi a fejlesztéskor ill. későbbi bővítésnél átláthatóbbá, valamint megkönnyíti a hibakeresést. Ilyen modul például a **/auth.routes.js** és **workout.routes.js**.

A **middleware**-ek a **backend/middleware/** mappában találhatók. Ezek feladata többek között az autentikáció ellenőrzése, a jogosultságkezelés, valamint a beérkező adatok validációja. A validációs folyamatok segítenek kiszűrni a hibás, hiányos vagy nem megfelelő formátumú adatokat, ezáltal csökkentik az adatbázisba irányuló káros **adattörlési** próbálkozások kockázatát is.

Az adatbázis-kapcsolat és a lekérdezések a **backend/sql/database.js** fájlban találhatók, MySQL adatbázis használatával. A szerver a **frontend/** mappából szolgálja ki a kliensoldali statikus fájlokat.

```
app.use(express.static(path.join(__dirname, '../frontend')));
```

2.2. ábra: statikus front kiszolgálás

```

    ///!API endpoints
    app.use('/', router);

    const endpoints = require('./api/api.js');
    app.use('/api', endpoints);

    ///! API 404
    app.use('/api', (req, res) => {
        return res.status(404).json({
            message: 'Az API végpont nem található!'
        });
    });

```

2.3. ábra: API mount

Főbb backend technológiák:

- express
- express-session
- mysql2/promise
- bcrypt
- request-ip
- dotenv
- crypto

2.2.2. Frontend

A frontend felépítése statikus **HTML** oldalakból, külön kezelt **CSS** fájlokból, **Bootstrap** alapú komponensekből és moduláris kliensoldali **JavaScript** állományokból áll.

A **HTML** fájlok határozzák meg az oldalak szerkezetét, a **CSS** fájlok a projekt egyedi stílusáért felelnek, míg a **Bootstrap** az alapvető reszponzív elrendezést, rácsrendszert és előre elkészített felületi elemek használatát támogatják. A **JavaScript** modulok az interakciók kezeléséért, az adatok betöltéséért, az **API**-kommunikációért, valamint az alapvető számítások, adatátalakítások és megjelenítési folyamatok végrehajtásáért felelnek.

A fontosabb frontend állományok elérési útvjai:

- frontend/html/*.html
- frontend/css/*.css

- frontend/javascript/*.js
- frontend/bootstrap/

A kliensoldali alkalmazás a backend által biztosított **REST API** végpontokon keresztül kommunikál a szerverrel. Az adatok lekérése, módosítása és törlése JavaScriptből indított **HTTP** kérésekkel történik. Az összes fetch függvény központosítva az **api.js** fájlban található, ezáltal egy fetch függv.-t többször is fel tudunk használni újra írás nélkül.

```
export async function registration(url, value){
  try {
    const data = await fetch(url, {
      method: 'POST',
      body: value
    })
    if(!data.ok){
      const res = await data.json();
      const err = new Error(res.message);
      switch(res.id){
        case 1:
          err.obj = null;
          break;
        case 2:
          err.obj = res.error[0];
          break;
        case 3:
          err.obj = res.error[0];
          break;
      }
      err.id = res.id;
      throw err;
    }
    return await data.json();
  } catch (error) {
    throw error;
  }
}
```

2.4. ábra: regisztráció POST fetch

```
export async function getFetch(url) {
  try {
    const data = await fetch(url, {
      method: 'GET',
      headers: {'Content-type': 'application/json'}
    })
    if(!data.ok){
      const res = await data.json();
      const err = new Error(res.message);
      err.error = res.error;
      throw err;
    }
    return await data.json();
  } catch (error) {
    throw error;
  }
}
```

2.5. ábra: GET fetch

2.2.3. Adatbázis

A rendszer **MySQL** adatbázist használ, amely a webalkalmazás működéséhez szükséges felhasználóval, edzéstervvel, receptekkel, statisztikával és adminisztrációval kapcsolatos adatokat tárolja. Az adatbázis szerkezetét a *backend/sql/database.sql* fájl hozza létre, amely tartalmazza a szükséges táblákat, mezőket, kapcsolatokat és alapértelmezett beállításokat.

Az adatbázis-kapcsolat a **backend/sql/database.js** fájlban van megvalósítva, **mysql2/promise** alapú connection pool használatával. A **pool** célja, hogy a backend több adatbázis-műveletet is hatékonyan tudjon kezelni, miközben az SQL-lekérdezéseket **aszinkron módon**, async/await szerkezettel futtatja.

2.2.3.1. Adatbázis-terv

A fő táblák:

- **users**: felhasználói fiókok, hitelesítési adatok, admin/aktív státusz, valamint az aktuálisan aktív edzésterv azonosítója
- **user_profiles**: kérdőív alapján rögzített demográfiai adatok, edzéscélok és személyes beállítások
- **user_weights**: felhasználói súlymérési előzmények tárolása

- **user_metrics**: BMI, BMR, TDEE, kalóriaszükséglet és fehérje-ajánlás tárolása
- **workout_plans, workout_days, day_exercises**: edzéstervek, edzésnapok és a napokhoz rendelt gyakorlatok modellezése
- **workout_calendar_logs, workout_calendar_exercises, workout_calendar_sets**: edzések naptár alapú ütemezése, naplózása és teljesítményadatainak tárolása
- **exercises**: gyakorlatok alapadatai, izomcsoport-besorolása és saját testsúlyos jelölése
- **foods, allergens, food_allergens**: recept-adatbázis, allergének és recept-allergén kapcsolatok kezelése
- **support_requests**: felhasználói hibajegyek és támogatási kérések kezelése
- **reset_tokens**: jelszó-visszaállításhoz használt tokenek tárolása
- **logs**: auditnaplók, adminműveletek és rendszeresemények tárolása
- **user_global_xp, user_muscle_xp, user_daily_stats, user_stats, user_exercise_prs**: edzésstatisztikák, XP rendszer, napi összesítések és személyes rekordok kezelése
- **user_friendships**: felhasználók közötti barátkapcsolatok és függőben lévő jelölések kezelése

2.2.3.2. Kulcsfontosságú entitások

A kulcsfontosságú entitások a rendszer fő funkciói alapján lettek létrehozva. Ezek az entitáscsoportok határozzák meg, hogy a felhasználói adatok, az edzéstervek, az edzésnaplók, a receptek, a statisztikák és a közösségi funkciók hogyan kapcsolódnak egymáshoz az adatbázisban.

2.2.3.2.1. Felhasználó és jelszó-visszaállítás

A **users** tábla a rendszer felhasználóinak alapadatait és jogosultsági állapotait tárolja. Ez az entitás több más funkcióhoz is kapcsolódik, például az edzéstervekhez, a profiladatokhoz, a jegyrendszerhez és a statisztikai adatokhoz.

A **users** tábla főbb mezői:

- **username**: a felhasználó egyedi felhasználóneve
- **email**: a felhasználó e-mail-címe
- **password_hash**: a jelszó titkosított formában tárolt értéke
- **registered**: a kérdőív kitöltési státusza
- **admin**: adminisztrátori jogosultság jelölése
- **active**: a felhasználói fiók állapota, amely alapján a felhasználó engedélyezett vagy tiltott
- **active_plan**: az aktuálisan aktív edzésterv azonosítója

```
CREATE TABLE IF NOT EXISTS users (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  
  first_name VARCHAR(100) NOT NULL,  
  last_name VARCHAR(100) NOT NULL,  
  
  username VARCHAR(50) NOT NULL UNIQUE,  
  email VARCHAR(150) NOT NULL UNIQUE,  
  password_hash VARCHAR(255) NOT NULL,  
  
  registered BOOLEAN NOT NULL DEFAULT FALSE,  
  admin BOOLEAN NOT NULL DEFAULT FALSE,  
  active BOOLEAN NOT NULL DEFAULT TRUE,  
  
  active_plan INT DEFAULT NULL,  
  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE  
  CURRENT_TIMESTAMP  
);
```

2.6. ábra: *users* tábla felépítése & létrehozása

Az **active_plan** mező az edzéstervhez tartozó azonosítót tárolja, így a rendszer ebből tudja meghatározni, hogy az adott felhasználónak jelenleg melyik edzésterve van használatban.

A **jelszó-visszaállítás** működéséhez a rendszer a **reset_tokens** táblát használja. Ebben a táblában tárolódnak az új jelszó beállításához szükséges visszaállítási tokenek, amelyek a felhasználóhoz kapcsolódnak. A tokenek hash formában kerülnek mentésre, így a rendszer nem közvetlenül olvasható tokenértéket tárol az adatbázisban.

A **reset_tokens** tábla fontosabb mezői:

- **user_id**: annak a felhasználónak az azonosítója, akihez a visszaállítási token tartozik
- **token_hash**: a visszaállítási token hash formában tárolt értéke
- **expires_at**: a token lejáratási ideje
- **used**: jelzi, hogy a token fel lett-e már használva

```
-- store reset-password token
CREATE TABLE IF NOT EXISTS reset_tokens (
  id INT AUTO_INCREMENT PRIMARY KEY,

  user_id INT NOT NULL,

  token_hash VARCHAR(64) NOT NULL,

  expires_at DATETIME NOT NULL DEFAULT (NOW() + INTERVAL 10
  MINUTE),

  used BOOLEAN NOT NULL DEFAULT FALSE,

  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

  UNIQUE(token_hash)
);
```

2.7. ábra: reset-tokens felépítése & létrehozása

2.2.3.2.2. Profil és mért adatok

A profil- és mért adatokhoz kapcsolódó táblák a felhasználó személyes, edzési és táplálkozási adatainak tárolására, valamint az ezekből számított egészségügyi mutatók kezelésére szolgálnak.

A **user_profiles** tábla tartalmazza a felhasználó kérdőívben megadott válaszokat, például a születési dátumot, magasságot, nemet, edzéscélt, tapasztalati szintet, edzési gyakoriságot, edzésheleyszint és étrendtípust. Ezek az adatok alapként szolgálnak a személyre szabott számításokhoz és ajánlásokhoz.

A **user_weights** tábla a felhasználó testsúlyadatait tárolja időbélyeggel együtt. Mivel egy felhasználóhoz több testsúly-bejegyzés is tartozhat, ez a tábla alkalmas a testsúly változásának későbbi nyomon követésére is.

A **user_metrics** tábla az aktuális profiladatok és a legfrissebb testsúly alapján számított értékeket tartalmazza. Ide kerülnek például a testtömegindex, az alapanyagcsere, a napi energiaigény, a célhoz igazított kalóriakeret és az ajánlott fehérjebevitel.

```
CREATE TABLE IF NOT EXISTS user_profiles (  
  id INT PRIMARY KEY,  
  
  birth_date DATE NULL,  
  height INT NULL,  
  
  gender ENUM('férfi', 'nő') NULL,  
  
  goal ENUM(  
    'tömegnövelés',  
    'szólkásítás',  
    'szinttartás'  
  ) NULL,  
  
  experience_level ENUM(  
    'kezdő (0-6 hónap)',  
    'középhaladó (6-24 hónap)',  
    'haladó (2+ év)'  
  ) NULL,  
  
  training_days ENUM(  
    '2-3 nap',  
    '4 nap',  
    '5-6 nap'  
  ) NULL,  
  
  training_location ENUM(  
    'konditeremben',  
    'otthon, súlyzókkal',  
    'otthon, saját testsúllyal'  
  ) NULL,  
  
  diet_type ENUM(  
    'mindenevő',  
    'vegetáriánus',  
    'vegán'  
  ) NULL,  
  
  meals_per_day ENUM(  
    '2-3 alkalom',  
    '4-5 alkalom',  
    '6 vagy több alkalom'  
  ) NULL,  
  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE  
    CURRENT_TIMESTAMP  
);
```

2.8. ábra: felhasználói profil tábla felépítése

A **user_metrics** főbb mezői:

- **bmi**: testtömegindex
- **bmr**: alapanyagcsere
- **tdee**: becsült napi energiafelhasználás
- **goal_calories**: célhoz igazított napi kalóriakeret
- **protein_recommended**: ajánlott napi fehérjebevitel

A számított értékek frissítése az aktuális profiladatok és testsúly alapján történik, így a rendszer a felhasználó adataiban bekövetkező változásokhoz igazodva tud pontosabb ajánlásokat adni.

```
CREATE TABLE IF NOT EXISTS user_weights (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  
  user_id INT NOT NULL,  
  weight DECIMAL(5,1) NOT NULL,  
  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

2.9. ábra: felhasználó súly tábla felépítés

2.2.3.2.3. Edzéstervek és naptár

Az edzéstervek és a naptár adatmodellje egymásra épülő táblákból áll. Az edzéstervek alapját az **exercises**, **workout_plans**, **workout_days** és **day_exercises** táblák adják. Ezek a táblák a gyakorlatok alapadatait, az edzéstervek alap adatait, a tervhez tartozó napokat, valamint az adott napokhoz rendelt gyakorlatokat tárolják.

A tervezési modell három fő szinten tárolja az edzésterv felépítését:

- **workout_plans**: az edzésterv alapadatai, például a felhasználó azonosítója, a terv neve, szintje, helyszíne, célja, publikus állapota és a napok száma
- **workout_days**: az edzéstervhez tartozó napok, például a nap sorszáma, neve, pihenőnap állapota és opcionális képe
- **day_exercises**: az egyes edzésnapokhoz rendelt gyakorlatok és azok sorrendje

```

CREATE TABLE IF NOT EXISTS workout_plans(
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT,
  name VARCHAR(100) NOT NULL,
  level ENUM('kezdő', 'közép', 'haladó') DEFAULT NULL,
  location ENUM('gym', 'home_weights', 'home_bodyweight') DEFAULT
  NULL,
  goal ENUM('tömeg', 'szalkasitas', 'szintentartas') DEFAULT NULL,
  description VARCHAR(255) DEFAULT NULL,
  is_public BOOLEAN DEFAULT FALSE,
  days_count TINYINT NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS workout_days(
  id INT AUTO_INCREMENT PRIMARY KEY,
  plan_id INT NOT NULL,
  day_number TINYINT NOT NULL,
  name VARCHAR(100) NOT NULL,
  isRestDay BOOLEAN DEFAULT FALSE,
  image_url VARCHAR(255) DEFAULT NULL
);

CREATE TABLE IF NOT EXISTS day_exercises(
  id INT AUTO_INCREMENT PRIMARY KEY,
  day_id INT NOT NULL,
  exercise_id INT NOT NULL,
  exercise_order INT NOT NULL
);

```

2.10. ábra: alap edzésterv táblák felépítése

Az **exercises** tábla külön kezeli a rendszerben elérhető gyakorlatokat. Minden gyakorlat rendelkezik névvel, izomcsoporttal és egy bodyweight mezővel, amely jelzi, hogy saját testsúlyos gyakorlatról van-e szó. Ez azért fontos, mert az edzésnapokhoz nem közvetlenül szöveges gyakorlatnevek kerülnek, hanem az **exercise_id** alapján hivatkozott gyakorlatok.

```
CREATE TABLE IF NOT EXISTS exercises(
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL UNIQUE,
  muscle_group ENUM(
    'mell',
    'hát',
    'váll',
    'bicepsz',
    'tricepsz',
    'alkar',
    'has',
    'ferde_has',
    'alsó_hát',
    'comb_első',
    'comb_hátsó',
    'farizom',
    'vádli',
    'teljes_test'
  ) NOT NULL,
  bodyweight BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

2.11. ábra: gyakorlatok tábla felépítése

Az aktív edzésterv napjainak dátum szerinti naplózása külön táblákban történik. Ez a rész már nem az edzésterv sablonját írja le, hanem azt, hogy egy adott felhasználó egy adott napon milyen edzést kapott, milyen állapotban van az edzés, és milyen teljesítményadatokat rögzített hozzá.

A felhasználó egy edzésterv kiválasztása után nemcsak magát a tervet menti el, hanem a backend előre legenerálja a következő két hónap naptári adatait is a kapcsolódó naptár táblákba. Ez azt jelenti, hogy a kiválasztott edzésterv napjai dátumokhoz kötve bekerülnek a **workout_calendar_logs** táblába, majd ezekhez kapcsolódóan létrejönnek a megfelelő gyakorlat- és sorozatbejegyzések is.

Ezen felül minden bejelentkezés után a backend ellenőrzi, hogy az adott felhasználóhoz tartozó naptárban meddig vannak előre legenerálva az edzésnapok. Amennyiben az utolsó legenerált nap kevesebb mint 7 napra van az aktuális dátumtól, a rendszer automatikusan újabb két hónapnyi naptári adatot generál. Ennek köszönhetően a felhasználó naptára folyamatosan előre feltöltve marad, anélkül hogy ezt manuálisan kellene kezelni.

A naptári edzésnaplózás három fő táblára épül:

- **workout_calendar_logs**: az adott felhasználóhoz tartozó naptári edzésnapokat tárolja, például az edzéstervet, az edzésnapot, a dátumot, a kezdési időt, az edzés időtartamát és az aktuális státuszt
- **workout_calendar_exercises**: a naptári edzésnaphoz tartozó konkrét gyakorlatokat tárolja, azok sorrendjével együtt
- **workout_calendar_sets**: az elvégzett sorozatok adatait tárolja, például a sorozatszámot, az ismétlésszámot és a használt súlyt

```
CREATE TABLE IF NOT EXISTS workout_calendar_logs(
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  workout_plan_id INT NOT NULL,
  workout_day_id INT NOT NULL,
  workout_date DATE NOT NULL,
  start_time DATETIME DEFAULT NULL,
  workout_time_sec INT NOT NULL DEFAULT 0,
  status ENUM('pending', 'started', 'completed', 'missed', 'rest')
  NOT NULL DEFAULT 'pending',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS workout_calendar_exercises(
  id INT AUTO_INCREMENT PRIMARY KEY,
  workout_calendar_log_id INT NOT NULL,
  exercise_id INT NOT NULL,
  exercise_order INT NOT NULL
);

CREATE TABLE IF NOT EXISTS workout_calendar_sets(
  id INT AUTO_INCREMENT PRIMARY KEY,
  workout_calendar_exercise_id INT NOT NULL,
  set_number INT NOT NULL,
  reps_done INT NOT NULL,
  weight_done DECIMAL(6,2) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

2.12. ábra: naptár törzs tábláinak felépítése

A **workout_calendar_logs** tábla státuszmezője alapján követhető, hogy az adott edzés még függőben van, elkezdődött, befejeződött, kimaradt vagy pihenőnapként szerepel. A **workout_calendar_exercises** és **workout_calendar_sets** táblák ehhez kapcsolódva tárolják a ténylegesen végrehajtott gyakorlatokat és azok teljesítményadatait.

Ez a felépítés lehetővé teszi, hogy az eredeti edzésterv szerkezete megmaradjon, miközben a naptárban külön naplózhatóak a ténylegesen elvégzett edzések. Ennek köszönhetően a rendszer egyszerre tudja kezelni az edzésterv sablonját, a naptári beosztást és az edzés közben rögzített teljesítményadatokat.

2.2.3.2.4. Recept- és jegyrendszer

A receptkezelés központi táblája a **foods**, amely a rendszerben elérhető ételek és receptek adatait tárolja. A tábla támogatja az admin által létrehozott, jóváhagyott recepteket, valamint a felhasználók által készített saját receptek kezelését is.

A recept létrehozójának típusa a **user_id** és a **created_by** mezők alapján különíthető el. Ha a receptet felhasználó hozta létre, akkor a **user_id** mező tartalmazza az adott felhasználó azonosítóját. Ha a receptet adminisztrátor hozta létre, akkor a **created_by** mező jelzi az admin azonosítóját. Ez lehetővé teszi, hogy a rendszer külön kezelje a felhasználói és az admin által rögzített recepteket.

A felhasználók saját receptjei alapértelmezetten nem jelennek meg más felhasználók számára. Ha egy felhasználó meg szeretné osztani a receptjét, akkor a **share** mező értéke igazra állítható. A recept azonban csak akkor válik mások számára is elérhetővé, ha az adminisztrátor jóváhagyja azt az **is_approved** mezővel. Így a rendszer biztosítja, hogy csak ellenőrzött receptek kerüljenek ki a publikus felületre.

A **foods** tábla fontosabb mezői:

- **id**: a recept egyedi azonosítója.
- **user_id**: a receptet létrehozó felhasználó azonosítója.
- **created_by**: a receptet létrehozó adminisztrátor azonosítója.
- **name**: a recept neve.
- **description**: a recept leírása, például az összetevők vagy az elkészítés menete.
- **share**: jelzi, hogy a felhasználó szeretné-e megosztani a receptet.
- **is_approved**: jelzi, hogy az admin jóváhagyta-e a receptet.
- **category**: a recept kategóriája.
- **calories_kcal**, **protein_g**, **carbs_g**, **fat_g**: a recept fő tápértékadatai.
- **goal_tag**, **diet_tag**: a recept cél és étrend szerinti besorolása.
- **high_protein**, **low_carb**, **bulk_friendly**, **cut_friendly**: ajánló és szűrési címkék.

```

CREATE TABLE IF NOT EXISTS foods (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT DEFAULT NULL,

  -- alap adatok
  name VARCHAR(150) NOT NULL,
  description TEXT NOT NULL,
  image_url VARCHAR(255) DEFAULT NULL,
  share BOOLEAN DEFAULT FALSE,

  category ENUM(...)
) NOT NULL,

  created_by INT DEFAULT NULL,
  is_approved BOOLEAN NOT NULL DEFAULT FALSE,

  -- tápértékek (100g / 100ml alap)
  calories_kcal DECIMAL(8,2) NOT NULL,
  protein_g DECIMAL(8,2) NOT NULL,
  carbs_g DECIMAL(8,2) NOT NULL,
  fat_g DECIMAL(8,2) NOT NULL,
  fiber_g DECIMAL(8,2) DEFAULT 0,
  sugar_g DECIMAL(8,2) DEFAULT 0,
  salt_g DECIMAL(8,2) DEFAULT 0,

  -- adagolás
  serving_size DECIMAL(8,2) NOT NULL,
  serving_unit ENUM(...)
) NOT NULL DEFAULT 'g',

  -- ajánló címkék
  goal_tag ENUM(...)
) NOT NULL DEFAULT 'mind',

  diet_tag ENUM(...)
) NOT NULL DEFAULT 'mindenevo',

  difficulty ENUM(...)
) NOT NULL DEFAULT 'konnyu',

  prep_time_min INT DEFAULT NULL,

  high_protein BOOLEAN NOT NULL DEFAULT FALSE,
  low_carb BOOLEAN NOT NULL DEFAULT FALSE,
  bulk_friendly BOOLEAN NOT NULL DEFAULT FALSE,
  cut_friendly BOOLEAN NOT NULL DEFAULT FALSE,

  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP
);

```

2.13. ábra: recept tábla felépítés

Az allergének külön entitásként az **allergens** táblában vannak tárolva. Ez azért hasznos, mert az allergénadatok egységesen kezelhetők, és nem kell minden receptnél külön szöveges formában ismételni őket.

Az **allergens** tábla fontosabb mezői:

- **id**: az allergén egyedi azonosítója.
- **name**: az allergén neve.

A receptek és allergének közötti kapcsolatot a **food_allergens** kapcsolótábla valósítja meg. Erre azért van szükség, mert egy recepthez több allergén is tartozhat, illetve ugyanaz az allergén több receptben is előfordulhat.

A **food_allergens** tábla fontosabb mezői:

- **food_id**: a recept azonosítója.
- **allergen_id**: az allergén azonosítója.

```
CREATE TABLE IF NOT EXISTS allergens (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL UNIQUE  
);  
  
CREATE TABLE IF NOT EXISTS food_allergens (  
  food_id INT NOT NULL,  
  allergen_id INT NOT NULL,  
  PRIMARY KEY (food_id, allergen_id)  
);
```

2.14. ábra: allergének tárolása és összekötés

A jegyrendszer működéséért a **support_requests** tábla felel. Ez tárolja a felhasználók által beküldött kapcsolatfelvételi üzeneteket, hibabejelentéseket és fejlesztési ötleteket. A tábla segítségével az adminisztrátorok megtekinthetik, kezelhetik, megválaszolhatják és lezárhatják a beérkező kéréseket.

A **support_requests** tábla fontosabb mezői:

- **id**: a támogatási jegy egyedi azonosítója.
- **user_id**: a jegyet létrehozó felhasználó azonosítója.
- **email**: a beküldő e-mail-címe.
- **category**: a jegy típusa, például kapcsolatfelvétel, hibabejelentés vagy ötlet.

- **subject**: a jegy tárgya.
- **message**: a felhasználó által beküldött üzenet.
- **admin_reply**: az adminisztrátor válasza.
- **status**: a jegy aktuális állapota.
- **related_request_id**: kapcsolódó korábbi jegy azonosítója.
- **replied_by_admin_id**: a válaszadó adminisztrátor azonosítója.

```
CREATE TABLE IF NOT EXISTS support_requests (
  id INT AUTO_INCREMENT PRIMARY KEY,

  user_id INT NOT NULL,
  email VARCHAR(150) NOT NULL,

  category ENUM('contact', 'bug', 'idea') NOT NULL,
  subject VARCHAR(150) NOT NULL,
  message TEXT NOT NULL,

  admin_reply TEXT DEFAULT NULL,

  status ENUM('open', 'seen', 'closed', 'closed_no_reply') NOT
  NULL DEFAULT 'open',

  related_request_id INT DEFAULT NULL,

  replied_by_admin_id INT DEFAULT NULL,
  replied_at TIMESTAMP NULL DEFAULT NULL,

  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
  CURRENT_TIMESTAMP
);
```

2.15. ábra: kapcsolatfelvétel törzs táblája

Ez a felépítés biztosítja, hogy a receptek, az allergének és a támogatási jegyek elkülönítve, mégis átláthatóan legyenek kezelhetők. A receptkezelésnél külön válik a létrehozás, a megosztás és az adminisztrátori jóváhagyás, míg a jegyrendszerben nyomon követhető a felhasználói bejelentések állapota és kezelése is.

2.2.3.2.5. Ranglista és barátok

A ranglista működése a **user_global_xp** táblára épül, amely felhasználónként tárolja az összesített **XP** értéket. Ez az érték határozza meg a felhasználó helyezését a barátok közötti privát ranglistán.

A barátkapcsolatok kezelését a **user_friendships** tábla biztosítja. Ebben a táblában tárolódik, hogy két felhasználó között milyen kapcsolat áll fenn, ki kezdeményezte a kapcsolatot, illetve hogy a kérés elfogadásra, elutasításra vagy blokkolásra került-e. A barátlista segítségével a felhasználók a saját ismerőseik között összehasonlíthatják fejlődésüket.

A **user_global_xp** tábla fontosabb mezői:

- **user_id**: a felhasználó egyedi azonosítója, egyben a tábla elsődleges kulcsa
- **xp**: a felhasználó összesített XP pontszáma
- **updated_at**: az XP érték utolsó módosításának időpontja

A **user_friendships** tábla fontosabb mezői:

- **user1_id**: a barátkapcsolat egyik felhasználójának azonosítója
- **user2_id**: a barátkapcsolat másik felhasználójának azonosítója
- **requested_by**: annak a felhasználónak az azonosítója, aki a barátkérést indította
- **status**: a kapcsolat aktuális állapota, például függőben lévő, elfogadott, elutasított vagy blokkolt
- **created_at**: a barátkérés létrehozásának időpontja
- **responded_at**: a barátkérésre adott válasz időpontja

```
CREATE TABLE IF NOT EXISTS user_friendships (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  
  user1_id INT NOT NULL,  
  user2_id INT NOT NULL,  
  requested_by INT NOT NULL,  
  
  status ENUM('pending', 'accepted', 'rejected', 'blocked') NOT  
  NULL DEFAULT 'pending',  
  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  responded_at TIMESTAMP NULL DEFAULT NULL  
);
```

2.16. ábra: barátrendszer tábla felépítése

2.2.3.2.6. Statisztikai adatok

A statisztikai adatok kezelését több külön tábla biztosítja, amelyek a felhasználók edzéseiből származó teljesítményadatokat, XP értékeket és személyes rekordokat tárolják. Ez a felépítés lehetővé teszi, hogy a rendszer **napi, összesített, izomcsoportos és gyakorlatonkénti** statisztikákat is meg tudjon jeleníteni.

A **napi** és **összesített** statisztikák a **user_daily_stats** és **user_stats** táblákban tárolódnak. A **user_daily_stats** egy adott naphoz kapcsolódó adatokat tartalmaz, például a teljesített edzések számát, az edzésvolument, az edzésidőt és az adott napon szerzett XP-t. A **user_stats** ezzel szemben a felhasználó teljes eddigi teljesítményét összegzi.

Az **XP**-alapú fejlődés követését a **user_muscle_xp** és **user_global_xp** táblák biztosítják. A **user_muscle_xp** izomcsoportonként tárolja a megszerzett **XP**-t, míg a **user_global_xp** a felhasználó összesített **XP** értékét tartalmazza. A gyakorlatonkénti személyes rekordok a **user_exercise_prs** táblában kerülnek tárolásra, ahol például a **legnagyobb használt súly** és a **legjobb volumen** is rögzíthető.

A **user_daily_stats** tábla fontosabb mezői:

- **user_id**: a felhasználó azonosítója
- **stat_date**: a napi statisztikai bejegyzés dátuma
- **completed_workouts**: az adott napon teljesített edzések száma
- **total_volume**: az adott napi összesített edzésvolumen
- **total_workout_time_sec**: az adott napon edzéssel töltött idő másodpercben
- **xp_gained**: az adott napon szerzett XP mennyisége

A **user_stats** tábla fontosabb mezői:

- **user_id**: a felhasználó egyedi azonosítója, egyben a tábla elsődleges kulcsa
- **completed_workouts**: a felhasználó összes teljesített edzésének száma
- **total_volume**: a felhasználó teljes edzésvolumene
- **pr_count**: az összes elért személyes rekord száma
- **total_workout_time_sec**: a teljes edzésidő másodpercben

A **user_exercise_prs** tábla fontosabb mezői:

- **user_id**: a felhasználó azonosítója
- **exercise_id**: annak a gyakorlatnak az azonosítója, amelyhez a személyes rekord tartozik
- **max_weight**: az adott gyakorlatban használt legnagyobb súly
- **best_volume**: az adott gyakorlatban elért legjobb volumen
- **achieved_at**: a személyes rekord elérésének időpontja

A **user_muscle_xp** tábla fontosabb mezői:

- **user_id**: a felhasználó azonosítója
- **muscle_group**: az XP-hez kapcsolódó izomcsoport
- **xp**: az adott izomcsoportoz tartozó XP érték

A **user_global_xp** tábla fontosabb mezői:

- **user_id**: a felhasználó egyedi azonosítója, egyben a tábla elsődleges kulcsa
- **xp**: a felhasználó összesített XP értéke
- **updated_at**: az XP érték utolsó módosításának időpontja

```

CREATE TABLE IF NOT EXISTS user_muscle_xp (
    user_id INT NOT NULL,
    muscle_group ENUM( ...
) NOT NULL,

    xp INT NOT NULL DEFAULT 0,

    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
    CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS user_global_xp (
    user_id INT PRIMARY KEY,
    xp INT NOT NULL DEFAULT 0,

    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
    CURRENT_TIMESTAMP,

    FOREIGN KEY (user_id)
        REFERENCES users(id)
        ON DELETE CASCADE
);

```

2.17. ábra: xp-t tároló táblák

```

CREATE TABLE IF NOT EXISTS user_daily_stats (
    user_id INT NOT NULL,
    stat_date DATE NOT NULL,

    completed_workouts INT NOT NULL DEFAULT 0,
    total_volume DECIMAL(12,2) NOT NULL DEFAULT 0,
    total_sets INT NOT NULL DEFAULT 0,
    total_reps INT NOT NULL DEFAULT 0,
    total_workout_time_sec INT NOT NULL DEFAULT 0,

    xp_gained INT NOT NULL DEFAULT 0,
    prs_achieved INT NOT NULL DEFAULT 0,

    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
    CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS user_stats (
    user_id INT PRIMARY KEY,

    completed_workouts INT NOT NULL DEFAULT 0,
    total_volume DECIMAL(12,2) NOT NULL DEFAULT 0,
    total_sets INT NOT NULL DEFAULT 0,
    total_reps INT NOT NULL DEFAULT 0,
    pr_count INT NOT NULL DEFAULT 0,
    total_workout_time_sec INT NOT NULL DEFAULT 0,

    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
    CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS user_exercise_prs (
    user_id INT NOT NULL,
    exercise_id INT NOT NULL,

    max_weight DECIMAL(6,2) NOT NULL DEFAULT 0,
    max_weight_reps INT DEFAULT NULL,
    best_volume DECIMAL(10,2) NOT NULL DEFAULT 0,

    achieved_at DATETIME DEFAULT NULL,

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
    CURRENT_TIMESTAMP
);

```

2.18. ábra: felhasználói statisztika tárolás

2.2.3.3. Adatmodell kiegészítések

Az adatbázis-séma létrehozása után több táblához külön kerülnek hozzáadásra az **idegen kulcsok**, **elsődleges kulcsok**, **egyedi megszorítások** és **ellenőrző feltételek**. Ezek határozzák meg a táblák közötti kapcsolatokat, és segítenek megőrizni az adatok konzisztenciáját.

A legtöbb kapcsolat **ON DELETE CASCADE** szabályt használ, így ha egy fő rekord törlésre kerül, akkor a hozzá tartozó alárendelt adatok is automatikusan törlődnek. Ez főleg a felhasználói adatoknál, edzésterveknél, edzésnaplóknál és statisztikáknál fontos, mert így nem maradnak az adatbázisban felesleges, árva rekordok.

Fontosabb kapcsolatok és megszorítások:

- **Felhasználói adatok:**

- A **user_profiles**, **user_metrics**, **user_global_xp**, **user_muscle_xp**, **user_daily_stats**, **user_exercise_prs**, **user_stats** és **reset_tokens** táblák a **users** táblához kapcsolódnak.
- A **user_weights** tábla a **user_profiles** táblához kötődik, mivel a testsúlyadatok a felhasználói profil részeként kezelhetők.
- A **user_muscle_xp**, **user_daily_stats** és **user_exercise_prs** táblák összetett kulcsokat használnak, hogy egy adott felhasználóhoz ne jöhessenek létre duplikált statisztikai rekordok.

- **Edzéstervek:**

- A **workout_plans** tábla a felhasználókhoz tartozó edzésterveket tárolja.
- A **users** tábla **active_plan** mezője az aktuálisan kiválasztott edzéstervre hivatkozik.
- A **workout_days** tábla az edzéstervek napjait tartalmazza.
- A **day_exercises** tábla az egyes napokhoz rendelt gyakorlatokat tárolja.
 - **Egyedi megszorítások** akadályozzák meg, hogy ugyanazon a napon ugyanaz a gyakorlat vagy ugyanaz a sorrendi pozíció többször szerepeljen.
 - Az **exercises** táblára mutató kapcsolat **ON DELETE RESTRICT** szabályt használ, ezért egy gyakorlat nem törölhető, ha még szerepel valamelyik edzéstervben.

- **Edzésnaplózás:**

- A **workout_calendar_logs** tábla az edzésnapló alapadatait tárolja.

- Ehhez kapcsolódnak a **workout_calendar_exercises** és **workout_calendar_sets** táblák, amelyek az adott edzés gyakorlatokat és sorozatait tartalmazzák.
- **Egyedi megszorítás** biztosítja, hogy egy felhasználónak egy adott dátumra csak egy edzésnaplója legyen.
- A további egyedi megszorítások megakadályozzák a gyakorlatok és sorozatszámok duplikálását egy edzésen belül.
- **Ételek és allergének:**
 - A **foods** tábla a recepteket és ételadatokat tárolja.
 - A **foods** rekordjai opcionálisan kapcsolódhatnak felhasználóhoz, ezért felhasználó törlésekor a hivatkozás **NULL** értékre áll, maga az étel viszont megmarad.
 - A **food_allergens** tábla köztes táblaként kapcsolja össze a **foods** és **allergens** táblákat, így egy ételhez több allergén is tartozhat.
- **Barátkapcsolatok:**
 - A **user_friendships** tábla a barátkapcsolatokat és barátkéréseket kezeli.
 - A **CHECK** feltételek megakadályozzák az önmagával létrehozott kapcsolatokat, illetve biztosítják, hogy ugyanaz a kapcsolat ne jöhessen létre fordított sorrendben is.
 - Az **egyedi megszorítás** biztosítja, hogy két felhasználó között csak egy kapcsolat szerepeljen.
- **Támogatási rendszer:**
 - A **support_requests** tábla a felhasználói kéréseket és az adminisztrátori válaszokat tárolja.
 - Kapcsolódik a **users** táblához, valamint önmagára is hivatkozhat a **related_request_id** mezőn keresztül.

- Adminisztrátor vagy kapcsolódó kérés törlésekor a hivatkozás **NULL** lesz, így a támogatási kérés továbbra is megmarad.

```
ALTER TABLE users
  ADD CONSTRAINT fk_users_active_workout_plan
  FOREIGN KEY (active_plan)
  REFERENCES workout_plans(id)
  ON DELETE SET NULL;

ALTER TABLE user_profiles
  ADD CONSTRAINT fk_user_profiles_users
  FOREIGN KEY (id)
  REFERENCES users(id)
  ON DELETE CASCADE;

ALTER TABLE user_metrics
  ADD CONSTRAINT fk_user_metrics_user
  FOREIGN KEY (user_id) REFERENCES users(id)
  ON DELETE CASCADE;

ALTER TABLE user_weights
  ADD CONSTRAINT fk_user_weights_users
  FOREIGN KEY (user_id)
  REFERENCES user_profiles(id)
  ON DELETE CASCADE;
```

2.18. ábra: táblák közti kapcsolat példa

2.3. Program komponensek

A MuscleMind programkomponensei úgy lettek felépítve, hogy az **edzésnaplózás**, **edzésterv-kezelés**, **receptkeresés**, **jegyrendszer**, **felhasználói profilok** és **adminisztráció** külön-külön, átlátható egységeken keresztül működjenek. A **backend feladata** a felhasználói műveletek kiszolgálása, az adatok ellenőrzése, az adatbázis-műveletek kezelése, valamint a jogosultságok vizsgálata.

A szerveroldali működés központi elemei a **server.js**, az **API útvonalak**, a **middleware fájlok** és a **database.js** adatbázis-kapcsolat. Ezek együtt biztosítják, hogy például egy edzés mentése, egy recept lekérése, egy felhasználói kérdőív feldolgozása vagy egy admin művelet **szabályozott és követhető folyamatként** történjen. A következő

alfejezetek ezeknek a komponenseknek a szerepét és működését mutatják be a MuscleMind rendszerén belül.

2.3.1. Futtatási környezet

A backend alkalmazás futtatásához szükséges parancsok a **backend/package.json** fájlban találhatók. A függőségek telepítése után a projekt fejlesztési módban **nodemon** segítségével, éles futtatás esetén pedig a **start** parancssal indítható.

Futtatáshoz használt parancsok:

- **npm install**
- **npm run dev** – fejlesztési indítás nodemon használatával
- **npm run start** – éles / normál indítás

A környezeti változók mintája a **backend/.env.example** fájlban található. A **Brevo** e-mail szolgáltatáshoz szükséges API-kulcsot a **BREVO_API_KEY** változóban kell megadni.

2.3.2. server.js

A **backend/server.js** az **Express alkalmazás belépési pontja**. Ebben a fájlban történik az alkalmazás példányosítása, az alap **middleware**-ek beállítása, az oldalútvonalak felvétele, az **API router** csatlakoztatása, a **hibakezelés** és a **szerver indítása**.

A fájl az alábbi **modulokat** és **saját middleware-eket** használja:

- **express**: Express alkalmazás és router létrehozása
- **express-session**: felhasználói munkamenetek kezelése
- **path**: fájlútvonalak összeállítása
- **kerdoiv.middleware.js**: kérdőív kitöltöttségének ellenőrzése
- **isAdmin.middleware.js**: admin jogosultság ellenőrzése
- **login.middleware.js**: bejelentkezéshez kötött oldalak védelme
- **database.js**: adatbázisfüggvények használata, például lejárt tokenek törlése

A szerver alapbeállításai:

- **IP-cím**: 127.0.0.1

- **Port:** 3000
- **JSON kérések kezelése:** `express.json()`
- **proxy mögötti működés engedélyezése:** `app.set('trust proxy', 1)`
- **HTTPS protokollal érkező kérések HTTP-re irányítása**
- **statikus frontend fájlok kiszolgálása** a `../frontend` könyvtárból

A **session-kezelés** **express-session** használatával történik. A session cookie **30** percreig érvényes, **httpOnly** beállítást használ, **sameSite** értéke **lax**, és a **secure** értéke **false**, mivel a projekt HTTP alapú működésre van beállítva. A **rolling: true** miatt aktív használat közben a session élettartama frissül.

A `server.js`-ben kezelt oldalútvonalak:

- **/:** főoldal, bejelentkezés és kitöltött kérdőív szükséges
- **/bejelentkezés:** bejelentkezési oldal, bejelentkezett felhasználó esetén átirányítás
- **/jelszo-keres:** jelszó-visszaállítási oldal, bejelentkezett felhasználó esetén átirányítás
- **/uj-jelszo:** új jelszó megadása
- **/regisztracio:** regisztrációs oldal, bejelentkezett felhasználó esetén átirányítás
- **/kerdoiv:** kérdőív oldal, bejelentkezés szükséges, már kitöltött kérdőív esetén átirányítás
- **/admin:** admin oldal, bejelentkezés és admin jogosultság szükséges
- **/profil:** profil oldal, bejelentkezés és kitöltött kérdőív szükséges
- **/kapcsolat:** kapcsolat oldal, bejelentkezés és kitöltött kérdőív szükséges
- **/tickets:** jegyrendszer oldal, bejelentkezés és kitöltött kérdőív szükséges
- **/ranglista:** ranglista oldal, bejelentkezés és kitöltött kérdőív szükséges
- **/naptar:** naptár oldal, bejelentkezés és kitöltött kérdőív szükséges
- **/etrend:** étrend oldal, bejelentkezés és kitöltött kérdőív szükséges
- **/edzesterv:** edzésterv oldal, bejelentkezés és kitöltött kérdőív szükséges
- **/statisztika:** statisztika oldal, bejelentkezés és kitöltött kérdőív szükséges

Az **API végpontok** külön fájlban találhatók. A `server.js` a **backend/api/api.js** állományt a **/api** útvonal alá csatlakoztatja: **`app.use('/api', endpoints)`**.

Hibakezelés:

- nem létező API végpont esetén **404-es JSON válasz** érkezik
- nem létező oldalútvonal esetén a **views/404.html** fájl töltődik be
- duplikált adatbázis-bejegyzés esetén **409-es JSON válasz** érkezik
- API hibák esetén **JSON válasz** kerül visszaküldésre
- oldalbetöltési hibák esetén a **views/500.html** fájl töltődik be

A szerver az **`app.listen`** segítségével indul el a **127.0.0.1:3000** címen, a fájl végén pedig egy időzített folyamat óránként meghívja a **`db.token_expire_del()`** függvényt, amely törli a **lejárt tokeneket** az adatbázisból.

2.3.3. API routing

A backend **API útvonalainak** központi meghívása a **backend/api/api.js** fájlban történik. Ez a fájl kapcsolja össze az **Express alkalmazással** az egyes funkcióterületekhez tartozó **route-modulokat**. Az **api.js** nem az egyes műveletek részletes megvalósításáért felel, hanem az **API szerkezetének központi pontjaként** működik.

Létrehozott **főbb API útvonalak**:

- **/api/auth** – bejelentkezés, regisztráció, kijelentkezés és munkamenet-kezelés
- **/api/question** – kérdőíves felhasználói adatok kezelése
- **/api/workout** – edzésterv, edzésnapok és gyakorlatok kezelése
- **/api/profile** – felhasználói profiladatok lekérése és módosítása
- **/api/tickets** – hibajegyek és felhasználói visszajelzések kezelése
- **/api/admin** – adminisztrációs műveletek
- **/api/stats** – statisztikai adatok lekérése
- **/api/meals** – receptek és étkezéshez kapcsolódó adatok kezelése
- **/api/ranglist** – ranglista és XP-alapú adatok kezelése

```
#!/Endpoints:
router.use('/auth', auth);
router.use('/question', qtn);
router.use('/workout', wOut);
router.use('/profile', profile);
router.use('/tickets', supportTicket);
router.use('/admin', adminApis);
router.use('/stats', statApis);
router.use('/meals', mealsApis);
router.use('/ranglist', ranglistApis);
```

2.19. ábra: API szerkezetének központi pontja

Az útvonalak **funkcióterületenként külön fájlban** vannak kezelve. Ez azt jelenti, hogy az **autentikáció**, a **profilkezelés**, az **edzéstervezés**, a **statisztika**, az **étkezéskezelés** és az **adminisztráció** logikája egymástól elkülönítve található meg a

backendben. Ez a felépítés átláthatóbbá teszi a projekt szerkezetét, mivel a kapcsolódó végpontok egy helyen maradnak.

Az API-végpontok működéséhez több esetben **middleware**-ek is kapcsolódnak. Ezek feladata a beérkező adatok **validálása**, a **munkamenet ellenőrzése**, valamint annak vizsgálata, hogy a felhasználó rendelkezik-e a szükséges **jogosultsággal** az adott művelet végrehajtásához. Ennek köszönhetően a route-fájlokban nem keveredik össze teljesen az adatok ellenőrzése és a konkrét kérelmfeldolgozás.

Például az **auth.routes.js** fájl az **autentikációhoz kapcsolódó végpontokat** kezeli. Ide tartozik a regisztráció, a bejelentkezés, a kijelentkezés, a felhasználónév lekérése, az admin jogosultság ellenőrzése, valamint a jelszó-visszaállítás folyamata.

A modulban a regisztrációs és bejelentkezési adatok ellenőrzése külön **middleware**-ekkel történik. A jelszavak **bcrypt** segítségével, **12 salt round** használatával kerülnek hashelésre. Sikeres regisztráció vagy bejelentkezés után a rendszer **session alapú munkamenetet** hoz létre, amelyben a felhasználó azonosítója és admin jogosultsága kerül eltárolásra.

A jelszó-visszaállítás **token alapú működésre** épül. A rendszer **crypto** segítségével generál token, majd annak **SHA-256 hash értékét** menti az adatbázisba. A token ellenőrzésekor a rendszer figyeli, hogy a token létezik-e, nem járt-e le, illetve nem lett-e már felhasználva. A visszaállító e-mail küldése a **Brevo API-n** keresztül történik.

Ezentúl a fontosabb műveletek, például a regisztráció, bejelentkezés, kijelentkezés és jelszómódosítás **naplózásra kerülnek**. Hiba esetén külön **hibalog** készül, amely segíti a fejlesztői hibakeresést is.

```

router.post('/registration', upload.none(), validateRegistration,
  async(request, response) => {
    try {
      const data = request.body;
      const ip = requestIp.getClientIp(request);
      const hashed = await bcrypt.hash(data.pass, saltRounds);
      const insert = Number(await db.registration_insert(data.
        firstN, data.lastN, data.userN, data.email, hashed));
      await db.log_id(insert, 'registration', 'registration 1/2',
        ip);
      request.session.user = {
        id: insert,
        admin: 0
      }
      response.status(201).json({
        message: 'Sikeres regisztráció'
      })
    } catch (error) {
      console.error(error.message)
      const ip = requestIp.getClientIp(request);
      try {
        await db.log_error('Server error - /registration', error.
          message, ip);
      } catch (error) {
        console.error('Logging failed:', error);
      }
      return response.status(500).json({
        message: 'Sikertelen elérés!'
      });
    }
  });
});

```

2.20. ábra: regisztráció POST api

2.3.4. Middleware

A backendben a middleware-ek az **API végpontok előtt futnak le**, és még a **vezérlőfüggvények meghívása előtt** ellenőrzik a beérkező kéréseket. Ezek a fájlok főként **validációs**, **session-kezelési** és **jogosultság-ellenőrzési** feladatokat látnak el.

A projektben használt fontosabb middleware fájlok:

- ➔ **login.middleware.js** – bejelentkezési adatok validációja, session ellenőrzés, védett oldalak és API végpontok kezelése
- ➔ **isAdmin.middleware.js** – admin jogosultság ellenőrzése
- ➔ **registration.middleware.js** – regisztrációs adatok ellenőrzése
- ➔ **kerdoiv.middleware.js** – kérdőív- és profiladatok validációja
- ➔ **workout.middleware.js** – edzésterv-adatok validációja

- **foods.middleware.js** – ételekhez és receptekhez kapcsolódó adatok ellenőrzése
- **ticket.middleware.js** – hibajegy-adatok validációja

A middleware réteg konkrét feladatai:

- Ellenőrzi, hogy a kötelező mezők szerepelnek-e a kérésben
- Kiszűri a nem engedélyezett vagy felesleges mezőket
- Reguláris kifejezésekkel ellenőrzi például az e-mail címek és jelszavak formátumát
- Megszakítja a hibás kéréseket, mielőtt azok elérnék az adatbázis-műveleteket
- Ellenőrzi, hogy a felhasználó rendelkezik-e aktív sessionnel
- Külön kezeli az oldalvédelmet és az API végpontok védelmét
- Admin funkcióknál ellenőrzi a megfelelő jogosultsági szintet

Példaként a **login.middleware.js** több, külön célra használt middleware függvényt tartalmaz. A **validateLogin** ellenőrzi, hogy a bejelentkezési kérés tartalmaz-e **e-mail címet és jelszót**, majd ezek formátumát **reguláris kifejezésekkel** vizsgálja. Ezután az **adatbázis-kezelő rétegen keresztül ellenőrzi**, hogy a megadott e-mail cím létezik-e a rendszerben. Hibás adat esetén a kérés nem jut tovább a bejelentkeztetést végző vezérlőfüggvényhez.

Ugyanebben a fájlban található **session alapú védelmi middleware-ek** is. A **redirectIfLoggedIn** a már bejelentkezett felhasználókat irányítja át, a **requireAuthPage** a védett HTML oldalak elérését korlátozza, míg a **requireAuthApi** az API végpontokat védi JSON válasszal. A **requestPassword** és **newPassword** függvények a jelszó-visszaállítás során érkező adatok **szerkezetét, típusát és formátumát** ellenőrzik.

Ezzel a felépítéssel az **útvonalkezelés átláthatóbb marad**, mert a route fájlokban csak a szükséges middleware-eket kell hozzárendelni az adott végponthoz, a részletes ellenőrzések pedig külön fájlokban maradnak.

```

async function validateLogin(req, res, next) {
  try {
    const { email, pass } = req.body;

    if(!email || !pass){
      return res.status(400).json({
        message: 'Hiányzó vagy érvénytelen mezők',
        id:1
      })
    }
    let patterError = [];
    if(!patternEmail.test(email)){
      patterError.push(1);
    }
    if(!patternPass.test(pass)){
      patterError.push(2);
    }
    if(patterError.length>0){
      return res.status(400).json({
        message: 'Érvénytelen adat(ok)',
        id: 2,
        error: patterError
      })
    }

    const emailExist = await db.email_exist(email);
    if(emailExist !== 1){
      return res.status(409).json({
        message: 'Érvénytelen e-mail cím vagy jelszó!',
        id: 3,
      })
    }

    next();
  } catch (error) {
    next(error)
  }
}

```

2.21. ábra: bejelentkezés validáció

```

function redirectIfLoggedIn(req, res, next){
  if(req.session?.user?.id){
    return res.redirect('/');
  }
  next()
}

function requireAuthPage(req, res, next){
  if(!req.session?.user?.id){
    return res.sendFile(path.join(__dirname, '../views/401.html'));
  }
  next()
}

function requireAuthApi(req, res, next){
  if(!req.session?.user?.id){
    return res.status(401).json({
      message: 'Nincs bejelentkezve!'
    });
  }
  next()
}

```

2.22. ábra: session vizsgálatok

2.3.5. Adatbázis szolgáltatások

A backend adatbázis-műveleteit a **database.js** fájl fogja össze. A modul a **mysql2/promise** csomaggal kapcsolódik a **MySQL adatbázishoz**, és **connection poolt** használ, ezért a lekérdezések nem külön-külön nyitnak új kapcsolatot, hanem a poolból kapnak elérhető kapcsolatot

database.js feladatai:

- **Kapcsolatkezelés:** MySQL connection pool létrehozása 10 darab maximális kapcsolattal.
- **Felhasználói adatok:** regisztráció, bejelentkezési adatok lekérése, e-mail és felhasználónév ellenőrzése.
- **Profiladatok:** kérdőívadatok, testsúly, személyes beállítások és számított metrikák mentése.
- **Edzéstervek:** saját és alapértelmezett edzéstervek, edzésnapok és gyakorlatok kezelése.
- **Edzésnaptár:** aktív terv beállítása, naptárbejegyzések generálása, edzések indítása, lezárása és halasztása.
- **Statisztikák:** volumen, XP, PR, napi statisztika, összesített statisztika és izomcsoportos XP frissítése.
- **Receptek:** ételek, allergének, felhasználói receptek, jóváhagyás és megosztási kérelem kezelése.
- **Support jegyek:** ticket létrehozása, listázása, státuszfrissítése és admin válasz mentése.
- **Admin műveletek:** felhasználók, receptek, edzéstervek, ticketek és napi statisztikai adatok kezelése.
- **Barátrendszer és ranglista:** barátjelölések, barátlista és XP-alapú ranglista lekérése.
- **Naplózás:** felhasználói események, bejelentkezések, regisztrációk és szerverhibák mentése.

A **több táblát érintő műveletek** egy része **tranzakcióban fut le**. Ilyen például az aktív edzésterv cseréje, az edzésnaptár generálása, a szettek mentése, az edzés halasztása, az admin edzéstervek létrehozása vagy módosítása, valamint az **edzésstatisztikák véglegesítése**. Hiba esetén a tranzakció **rollbackkel visszavonja** a részleges módosításokat.

```

async function saveUserMetrics(userId) {
  const metrics = await calculateUserMetrics(userId);

  const sql = `
    INSERT INTO user_metrics (
      user_id,
      bmi,
      bmr,
      tdee,
      goal_calories,
      protein_recommended
    ) VALUES (?, ?, ?, ?, ?, ?)
    ON DUPLICATE KEY UPDATE
      bmi = VALUES(bmi),
      bmr = VALUES(bmr),
      tdee = VALUES(tdee),
      goal_calories = VALUES(goal_calories),
      protein_recommended = VALUES(protein_recommended)
  `;

  const [result] = await pool.execute(sql, [
    userId,
    metrics.bmi,
    metrics.bmr,
    metrics.tdee,
    metrics.goalCalories,
    metrics.proteinRecommended
  ]);

  return result.affectedRows;
}

```

2.23. ábra: user metrics elmentése vagy frissítése

2.4. Algoritmusok

Ebben a részben azok a háttér folyamatok kerülnek bemutatásra, amelyek a felhasználói adatok feldolgozását és az alkalmazás automatikus döntéseit végzik.

A rendszer algoritmusokat használ többek között a hitelesítésnél, a kérdőív válaszaiból számított metrikáknál, az edzésterv létrehozásánál, a naptár generálásánál és átütemezésénél, valamint az edzések lezárása után frissülő statisztikáknál. Ezek mellett ide tartozik a ranglista, a barátokhoz kapcsolódó adatok és a receptek kezelési logikája is.

2.4.1. Auth, jelszó és tokenkezelés

A regisztráció során a rendszer a felhasználó által megadott jelszót **nem nyers formában tárolja**. A jelszó hash-elését a **bcrypt.hash()** végzi, az eredmény pedig a **password_hash** mezőbe kerül mentésre. Bejelentkezéskor a megadott jelszó ellenőrzése a **bcrypt.compare()** függvénnyel történik, amely a beküldött jelszót hasonlítja össze az adatbázisban tárolt hash értékkel.

Bejelentkezés utáni ellenőrzések és műveleteket:

- ellenőrzi, hogy létezik-e felhasználó a megadott email címmel
- elutasítja a belépést, ha a fiók blokkolt állapotú, vagyis **active = 0**
- sikeres hitelesítés után **session-t hoz létre**
- naptárfrissítést indít, ha az aktuálisan aktív edzéstervből **kevesebb mint 7 nap** van hátra

A jelszó visszaállítás során a rendszer egy **egyszer használható token** generál a következő módon:

```
crypto.randomBytes(32).toString('base64url')
```

A generált token nyers formában **nem kerül eltárolásra az adatbázisban**. A rendszer először **SHA-256** algoritmussal hash-eli, majd ezt a hash értéket menti el a **reset_tokens** táblába. A tokenhez **10 perces lejárat** idő tartozik, így a jelszó visszaállítási link csak rövid ideig használható fel.

A lejárt tokenek törlését a **token_expire_del()** folyamat végzi, amely **óránként** fut le, és eltávolítja az érvénytelen rekordokat a **reset_tokens** táblából.

```

router.post('/new-password', loginMw.newPassword, async(request,
response)=>{
  try {
    const ip = requestIp.getClientIp(request);
    const { password, token} = request.body;

    ///? token check
    const token_hash = crypto.createHash('sha256').update(token).
    digest('hex');
    const tokenData = await db.find_token(token_hash);
    if (!tokenData) {
      return response.status(400).json({
        message: 'Érvénytelen vagy lejárt link.'
      });
    }
    if (new Date(tokenData.expires_at) < new Date()) {
      return response.status(400).json({
        message: 'Érvénytelen vagy lejárt link.'
      });
    }
    if (tokenData.used) {
      return response.status(400).json({
        message: 'Érvénytelen vagy lejárt link.'
      });
    }
    const newPass = await bcrypt.hash(password, saltRounds);
    await db.update_password(tokenData.user_id, newPass);
    await db.set_used(tokenData.id);
    await db.log_id(tokenData.user_id, 'Password update',
    'Successful update!', ip);

    return response.status(200).json({
      message: 'Sikeres jelszó változtatás!'
    });
  } catch (error) {

```

2.24. ábra: jelszó visszaállítás - új jelszó mentése

```

// óránként lejárt tokenek törlése
setInterval(async () => {
  try {
    await db.token_expire_del();
  } catch (error) {
    console.error('Cleanup error:', error.message);
  }
}, 60 * 60 * 1000);

```

2.25. ábra: óránként lejárt token törlés

```

///? interval delete expired tokens
async function token_expire_del() {
  const del = 'DELETE FROM reset_tokens WHERE expires_at < NOW()
  OR used = TRUE';
  const [rows] = await pool.execute(del);
  return rows;
}

```

2.26. ábra: token_expire_del query

2.4.2. Kérdőív és metrika számítás

A rendszer a kérdőív során megadott adatokat a **user_profiles** és **user_weights** táblákban tárolja. A **saveUserMetrics(userId)** metódus feladata, hogy a felhasználó profiladatai és aktuális testsúlya alapján kiszámítsa és elmentse a főbb egészségügyi és étrendi metrikákat.

A metódus működése a következő lépésekből áll:

- **Életkor** kiszámítása a **birth_date** mező alapján
- **BMI** kiszámítása a **testsúly** és **testmagasság** alapján
- **BMR** kiszámítása **nem szerint eltérő képlettel**
- **Aktivitási szorzó** meghatározása a **heti edzésszám** alapján
- **TDEE** kiszámítása a **BMR** és az **aktivitási szorzó** alapján
- **Célkalória** meghatározása a **felhasználó célja szerint**: tömegnövelés, szálkásítás vagy szintentartás
- **Ajánlott napi fehérjebevitel** kiszámítása a testsúly alapján

A számítási logika több kisebb függvényre van bontva. A **getActivityMultiplier()** az aktivitási szorzót határozza meg, a **getGoalCalories()** a célhoz igazított napi kalóriamennyiséget számolja ki, míg a **getProteinRecommended()** az ajánlott napi fehérjebevitelet adja vissza.

A **metrikák frissítése** automatikusan történik a **kérdőív kitöltése után**, illetve minden olyan fontosabb **profiladat módosításakor**, amely befolyásolja a számított értékeket. Ilyen adat lehet például a **testsúly**, **testmagasság**, **születési dátum**, **nem**, **heti edzésszám** vagy a **felhasználó aktuális célja**. Ezek változtatása után a rendszer **újraszámolja és frissíti** a BMI, BMR, TDEE, célkalória és ajánlott napi fehérjebevitel értékeit, így az ajánlások **mindig az aktuális felhasználói adatok alapján** készülnek.

```

async function calculateUserMetrics(userId) {
  const profileSql = `...
  `;

  const [profileRows] = await pool.execute(profileSql, [userId]);
  const profile = profileRows[0];

  if (!profile) {...
  }

  const weightData = await getUserWeightData(userId);

  if (!weightData) {...
  }

  if (!profile.birth_date || !profile.height || !profile.gender ||
    !profile.goal || !profile.training_days) {
    throw new Error('Hiányos profil adatok a számításhoz.');
```

2.27. ábra: felhasználó mutatószámokat kiszámító függv.

2.4.3. Edzésterv generálás és naptár

Az edzésterv generálásában majd annak naptárba való illesztését a `workout.routes.js` és `database.js` közösen valósítja meg.

2.4.3.1. Új felhasználói edzésterv létrehozása

Az új edzésterv létrehozását a **POST /api/workout/newPlan** végpont kezeli. A végpont **csak bejelentkezett felhasználó** számára érhető el, a beküldött adatok ellenőrzését pedig a **validateNewPlan** middleware végzi.

A kérés feldolgozásakor a rendszer lekéri a bejelentkezett felhasználó azonosítóját a **session adatokból**, majd adatbázis-kapcsolatot nyit és **tranzakciót indít**. Ez azért fontos, mert az edzésterv több egymáshoz kapcsolódó adatból áll, ezért a mentésnek egységesen kell sikerülnie.

A létrehozási folyamat fő lépései:

- az **edzésterv alaprekordjának létrehozása** a terv nevével, a felhasználó azonosítójával és a napok számával,
- a **beküldött edzésnapok bejárása**,
- az egyes napok külön mentése a nap sorszámaival, nevével és **pihenőnap státuszával**,
- nem pihenőnap esetén a kiválasztott **gyakorlatok hozzárendelése** az adott naphoz,
- a gyakorlatok **sorrendjének eltárolása** a napon belül.

Sikeres mentés esetén a rendszer **commit művelettel véglegesíti** a tranzakciót, majd **WORKOUT_PLAN_CREATE** eseményként naplózza az új edzésterv létrehozását. A kliens ilyenkor **201-es státuszkódot** és sikeres létrehozást jelző választ kap.

Hiba esetén a rendszer **rollback művelettel** visszavonja a teljes mentési folyamatot. Ennek köszönhetően nem marad **részlegesen létrehozott edzésterv** az adatbázisban. A sikertelen művelet **ERR_WORKOUT_PLAN_CREATE** eseményként kerül naplózásra, a kliens pedig **500-as státuszkódú választ** kap.

```

router.post('/newPlan', requireAuthApi, validateNewPlan, async
(request, response)=>{
  const conn = await db.pool.getConnection();

  try {
    await conn.beginTransaction();

    const workoutPlan = request.body;
    const userId = request.session.user.id;
    const ip = requestIp.getClientIp(request);
    const planId = await db.createWorkoutPlan(
      conn,
      userId,
      workoutPlan.name.trim(),
      workoutPlan.days.length
    );
    for (let i = 0; i < workoutPlan.days.length; i++) {
      const day = workoutPlan.days[i];
      const dayId = await db.createWorkoutDay(
        conn,
        planId,
        day.dayNumber,
        day.name.trim(),
        day.restDay
      );
      if (!day.restDay) {
        for (let j = 0; j < day.exercises.length; j++) {
          const exercise = day.exercises[j];
          await db.createDayExercise(
            conn,
            dayId,
            exercise.exerciseId,
            exercise.order
          );
        }
      }
    }

    await conn.commit();

    await db.log_id(
      userId,
      'WORKOUT_PLAN_CREATE',
      `Új edzésterv létrehozva. planId: ${planId}`,
      ip
    );

    return response.status(201).json({
      message: 'Edzésterv sikeresen létrehozva.'
    });
  } catch (error) {

```

2.28. ábra: új edzésterv létrehozása

2.4.3.2. Naptárgenerálás

A naptárgenerálás két esetben indul el: amikor a felhasználó új aktív edzéstervet választ, vagy amikor az aktuális aktív tervből kevesebb mint 7 napnyi jövőbeli naptárbejegyzés maradt. Ilyenkor a rendszer az aktív edzésterv napjai és gyakorlatai alapján két hónapos időtartamra tölti fel a naptárat.

A naptár frissen tartását a **calendarUpToDate(userId)** algoritmus végzi. Ez a **GET /api/workout/calendar** végpont meghívásakor fut le, mielőtt a rendszer visszaadná a felhasználó naptáradatait, illetve minden bejelentkezés után.

Az algoritmus először ellenőrzi a mai naphoz tartozó edzés **státuszát**. Ha a mai napi edzés már **completed** állapotú, akkor az új naptárgenerálás **nem írja felül az adott napot**, hanem csak a következő naptól indul. Ellenkező esetben a generálás a mai nappal kezdődik.

A generálás fő lépései:

- a rendszer lekéri a mai naphoz tartozó edzés **státuszát**,
- ha a mai nap státusza **completed**, akkor a kezdődátum a következő nap lesz,
- a kezdődátumtól törli a **jövőbeli naptárbejegyzéseket** és a hozzájuk tartozó gyakorlatokat,
- **két hónapos időtartamra** új napi logokat generál,
- pihenőnap esetén a bejegyzés **rest** státuszt kap,
- edzésnap esetén a bejegyzés **pending** státusszal jön létre,
- az adott naphoz tartozó gyakorlatokat átmásolja a `day_exercises` táblából a naptárhoz kapcsolódó gyakorlatlistába.

A két hónapos generálási időtartam miatt a **felhasználó előre látja a következő edzésnapjait**, miközben az adatbázisban csak a szükséges jövőbeli bejegyzések jönnek létre. A korábbi, már **lezárt edzések így nem módosulnak**, a jövőbeli napok viszont az aktuálisan kiválasztott tervhez igazodnak.

Ez azért szükséges, mert a naptár nemcsak az **aktív terv kiválasztásakor** jön létre, hanem használat közben is **folyamatosan karbantartásra kerül**. Így a felhasználónak mindig rendelkezésére állnak a **közelgő edzésnapok**, anélkül hogy manuálisan újra kellene generálnia a naptárat.

```

router.get('/calendar', requireAuthApi, async (request, response) => {
  try {
    const userId = request.session.user.id
    // 2 honapra előre generalas
    await db.calendarUpToDate(userId)
    const rows = await db.getUserCalendar(userId)

    // napok csoporthoz
    const calendarMap = {}
    for (const row of rows) {
      // datum konvertalasa
      const workoutDate = new Date(row.workout_date)
      const yearStr = workoutDate.getFullYear()
      let monthStr = workoutDate.getMonth() + 1
      let dayStr = workoutDate.getDate()

      if (monthStr < 10) {
        monthStr = '0' + monthStr
      }
      if (dayStr < 10) {
        dayStr = '0' + dayStr
      }

      const dateStr = yearStr + '-' + monthStr + '-' + dayStr

      if (!calendarMap[dateStr]) {
        calendarMap[dateStr] = {
          date: dateStr,
          log_id: row.log_id,
          dayName: row.day_name,
          isRestDay: row.isRestDay,
          status: row.status,
          exercises: []
        }
      }

      if (row.exercise_name) {
        calendarMap[dateStr].exercises.push({
          calendar_exercise_id: row.calendar_exercise_id,
          name: row.exercise_name,
          muscle_group: row.exercise_muscle_group,
          order: row.exercise_order
        })
      }
    }

    return response.status(200).json({
      calendar: Object.values(calendarMap)
    })
  } catch (error) {

```

2.29. ábra: naptár adatainak lekérése

2.4.3.3. Naptár-átütemezés

Az edzés átütemezését a **PATCH /workout/calendar/postpone/:logId** végpont kezeli. A végpont a kérésben kapott **logId** alapján azonosítja az átütemezendő naptárbejegyzést, az új dátumot pedig a kérés törzséből olvassa ki **newDate** mezőként.

A kérés feldolgozása előtt a rendszer ellenőrzi, hogy az új dátum meg lett-e adva. Amennyiben a **newDate** hiányzik, a végpont **400-as státuszkóddal** tér vissza.

Az átütemezés tényleges logikáját a **postponeWorkoutCalendarLog(userId, logId, newDate)** adatbázisfüggvény végzi. A függvény két nap közti csere logikát használ, ha a cél dátumon már található edzés.

A **cserefolyamat** fő lépései:

- a rendszer megkeresi az átütemezendő edzést a **logId** és a **userId** alapján,
- ellenőrzi, hogy a cél dátumon van-e már másik edzés,
- ha a cél dátum szabad, az edzés dátumát közvetlenül az új dátumra módosítja,
- ha a cél dátumon már van edzés, a rendszer egy **ideiglenes tartalék dátumot** használ,
- az ideiglenes dátum segítségével felcseréli a két naptárbejegyzés dátumát,
- ezzel elkerüli a dátumokra vonatkozó **UNIQUE duplikációs hibát**

Sikeres átütemezés esetén a kliens **200-as státuszkódot** és sikeres halasztást jelző választ kap. Hiba esetén a végpont **500-as státuszkóddal** tér vissza.

```

async function postponeWorkoutCalendarLog(userId, logId, newDate) {
  const connection = await pool.getConnection();
  try {
    await connection.beginTransaction();

    // lekerjük a jelenlegi edzést, hogy megtudjuk a dátumot
    const [currentLog] = await connection.execute(...);

    if (currentLog.length === 0) {
      throw new Error('Nem található az edzés!');
    }

    const currentDate = formatDate(new Date(currentLog[0].workout_date));

    // megnezzük van-e a cél dátumon másik edzés
    const [targetLog] = await connection.execute(...);

    if (targetLog.length > 0) {
      const targetLogId = targetLog[0].id;

      // létrehozunk egy ideiglenes dátumot ('1000-01-01') a
      // csereledéshez hogy ne legyen duplicate hiba
      await connection.execute(
        `UPDATE workout_calendar_logs SET workout_date =
        '1000-01-01' WHERE id = ?`,
        [targetLogId]
      );

      // atmozgatjuk a mostani edzést az új dátumra
      await connection.execute(
        `UPDATE workout_calendar_logs SET workout_date = ?
        WHERE id = ?`,
        [newDate, logId]
      );

      // végül a régi dátumra tesszük a másikat
      await connection.execute(
        `UPDATE workout_calendar_logs SET workout_date = ?
        WHERE id = ?`,
        [currentDate, targetLogId]
      );
    } else {
      // ha nincs a cél dátumon semmi, csak simán atallítjuk
      await connection.execute(
        `UPDATE workout_calendar_logs SET workout_date = ?
        WHERE id = ?`,
        [newDate, logId]
      );
    }
  }
}

```

2.30. ábra: edzésterv halasztó függv.

2.4.4. Edzés lezárása és statisztikák

Az edzés lezárását a **PATCH /calendar/finish/:logId** végpont indítja el. A végpont a **logId** paraméterből azonosítja a lezárandó edzésnaplót, a bejelentkezett felhasználót pedig a **sessionben tárolt request.session.user.id** alapján határozza meg. Ezeket az adatokat adja át a **finalizeWorkoutStats(userId, logId)** függvénynek, amely a tényleges statisztikai feldolgozást végzi.

A lezárás előtt a rendszer ellenőrzi, hogy az adott edzésnapló **korábban nem lett-e már véglegesítve**. Erre azért van szükség, mert ugyanannak az edzésnek a többszöri lezárása **duplikált XP-t, hibás napi statisztikát és többszörösen számolt izomcsoport-terhelést eredményezne**. Ha a rendszer azt érzékeli, hogy az edzés már véglegesítve lett, a folyamat hibával megszakad, az API pedig 400-as státuszkóddal válaszol.

A lezárási folyamat főbb lépései:

- A végpont kiolvassa a lezárandó edzésnapló azonosítóját a **logId** URL-paraméterből.
- A rendszer meghatározza a bejelentkezett felhasználót a sessionben tárolt **userId** alapján.
- A **finalizeWorkoutStats(userId, logId)** függvény elindítja az edzéshez kapcsolódó statisztikai feldolgozást.
- Ellenőrzi, hogy az adott edzésnapló nem lett-e már korábban lezárva.
- Lekéri az edzéshez tartozó gyakorlatokat és a felhasználó által rögzített sorozatadatokat.
- A **workout_calendar_sets** táblában tárolt ismétlésszámok és súlyértékek alapján kiszámolja a teljesített volument.
- Saját testsúlyos gyakorlat esetén a felhasználó testsúlyát is beszámítja a volumenbe.
- A számított volument **izomcsoportonként összesíti**.
- Ellenőrzi, hogy az adott gyakorlatoknál született-e új egyéni rekord.
- **Új vagy jobb rekord esetén** beszúrja vagy frissíti a **user_exercise_prs** tábla megfelelő rekordját.
- A teljesített volumen alapján **XP-t** számol.
- Frissíti vagy létrehozza az adott naphoz tartozó **user_daily_stats** rekordot.
- Növeli a felhasználó **összesített XP** értékét.
- Frissíti az érintett **izomcsoportokhoz** tartozó **XP** értékeket.
- Frissíti az összesített felhasználói statisztikákat a **user_stats** táblában.
- Sikeres feldolgozás után az edzésnapló **véglegesített állapotba kerül**.

A függvény a sorozatok alapján számolja ki az edzés volumenét. A számítás alapját a **workout_calendar_sets** táblában tárolt ismétlésszámok és súlyértékek adják.

Súlyzós gyakorlat esetén: **súly $\times$ ismétlésszám**

Saját testsúlyos gyakorlat esetén: **(testsúly + plusz súly) $\times$ ismétlésszám**

Az így kiszámolt volumen nemcsak gyakorlatonként kerül felhasználásra, hanem **izomcsoportonként is összesítésre kerül**. Ez alapján a rendszer meg tudja határozni, hogy az adott edzés mely izomcsoportokat milyen mértékben terhelte.

A lezárási folyamat része az egyéni rekordok kezelése is. A rendszer minden érintett gyakorlatnál ellenőrzi, hogy az aktuális edzés során teljesített érték meghaladja-e a felhasználó korábbi rekordját.

Az egyéni rekord kezelése során:

- ha a felhasználónak még nincs rekordja az adott gyakorlatra, új rekord jön létre a **user_exercise_prs** táblában.
- ha már létezik rekord, de az aktuális teljesítmény jobb, a **meglévő rekord frissül**.
- ha az aktuális teljesítmény nem haladja meg a korábbi rekordot, a rekord nem változik.

A volumen alapján kerül kiszámításra az **XP** értéke. A számítás alapja: **Math.floor(volume / 10)**. Ez azt jelenti, hogy minden 10 egységnyi teljesített volumen után 1 XP jár. Az így kapott XP több statisztikai rekord frissítésénél is szerepet kap.

Az **XP és statisztikai adatok frissítése** során:

- frissül az adott naphoz tartozó **user_daily_stats** rekord;
- ha az adott naphoz még nincs napi statisztika, a rendszer új rekordot hoz létre;
- növekszik a **felhasználó összesített XP** értéke;
- frissülnek az **érintett izomcsoportok XP** értékei;
- módosulnak a **user_stats** táblában tárolt összesített statisztikák.

Sikeres végrehajtás esetén az API „**Edzés lezárva és statisztikák frissítve!**” választ adja vissza.

Hiba esetén a végpont külön kezeli azt az esetet, amikor az edzés már korábban véglegesítve lett vagy duplikáció történne. Ilyenkor **400-as státuszkódot** ad vissza a konkrét hibaüzenettel. Egyéb szervertoldali hiba esetén **500-as státuszkóddal** válaszol.

```
async function finalizeWorkoutStats(userId, logId) {
  const connection = await pool.getConnection();
  try {
    await connection.beginTransaction();

    // log betöltése és ellenőrzése
    const [logRows] = await connection.execute(
      `SELECT
        status,
        DATE_FORMAT(workout_date, '%Y-%m-%d') AS workout_date
      FROM workout_calendar_logs
      WHERE id = ? AND user_id = ?`,
      [logId, userId]
    );

    if (logRows.length === 0) {
      throw new Error("Edzés nem található!");
    }
    if (logRows[0].status === 'completed') {
      throw new Error("Ez az edzés már véglegesítve lett (duplikáció védelem)!");
    }

    const formattedWorkoutDate = logRows[0].workout_date;

    // edzés lezárása és időtartam kiszámolása.
    await connection.execute(
      "UPDATE workout_calendar_logs SET status = 'completed',
      workout_time_sec = TIMESTAMPTDIFF(SECOND, start_time, NOW
      ()) WHERE id = ?",
      [logId]
    );

    // lekerjük a logból a friss workout_time_sec adatot
    const [timeRows] = await connection.execute('SELECT
      workout_time_sec FROM workout_calendar_logs WHERE id = ?',
      [logId]);
    const workoutTimeSec = timeRows.length > 0 ? (timeRows[0].
      workout_time_sec || 0) : 0;

    // felhasználó aktuális súlya
    const userWeight = await getLatestWeight(userId)

    // mentett szettek beolvasása (workout_calendar_sets +
    // exercises.muscle_group + bodyweight)
    const [sets] = await connection.execute(`
      SELECT
        s.id as set_id,
        s.weight_done AS weight,
        s.reps_done AS reps,
        e.id as exercise_id,
        e.muscle_group,
        e.bodyweight`
  )
}
```

2.31. ábra: lezárás után statisztika számító függv.

2.4.5. Ranglista és barátok

A ranglista adatait a backend a **getLeaderboard(userId)** függvényen keresztül kéri le. A lekérdezés csak a felhasználót és azokat a felhasználókat jeleníti meg, akikkel **elfogadott baráti kapcsolatban áll**. Így a ranglista **nem globális**, hanem a felhasználó saját kapcsolati körére épül.

A ranglista összeállításánál a rendszer a **felhasználók XP** értékeit veszi alapul. A lekérdezés **bal oldali csatlakozást használ**, ezért azok a felhasználók is megjelenhetnek a listában, akikhez még nem tartozik külön XP rekord. Ilyen esetben az érték **0 XP**-ként kerül visszaadásra, így a frontend **egységesen tudja kezelni** a ranglista elemeit.

A **baráti kapcsolatokat** a rendszer **egységes ID-sorrendben** tárolja, ezért ugyanaz a kapcsolat **nem kerülhet be kétszer** fordított irányban. Például a 3-as és 8-as felhasználó kapcsolata mindig ugyanabban a formában kerül mentésre, **függetlenül attól, ki küldte a jelölést**.

A működés főbb lépései:

- A kliens minden ranglista- és barátkezelési műveletet hitelesített API-végponton keresztül ér el.
- A **getLeaderboard(userId)** csak a bejelentkezett felhasználót és az elfogadott barátait adja vissza.
- A **hiányzó XP** értékek **0 XP-ként** kerülnek kezelésre.
- A **searchUsers** függvény **LIKE** operátorral keres, és limitált számú találatot ad vissza.
- Barátjelölés küldése előtt a rendszer ellenőrzi, hogy létezik-e már kapcsolat vagy függőben lévő jelölés.
- A **user_friendships** tábla a kisebb-nagyobb felhasználói ID sorrendet használja a duplikált kapcsolatok elkerülésére.
- **Elfogadás, elutasítás és törlés** esetén a backend ellenőrzi, hogy az adott kapcsolat az aktuálisan bejelentkezett felhasználóhoz tartozik-e.

A **barátjelölés küldésekor** a rendszer először ellenőrzi, hogy a két felhasználó között **létezik-e már kapcsolat vagy függőben lévő jelölés**. Ha igen, a

backend nem hoz létre új rekordot. **Elfogadás, elutasítás és törlés** esetén a művelet csak akkor hajtható végre, ha az adott kapcsolat valóban az aktuálisan bejelentkezett felhasználóhoz tartozik. Ez megakadályozza, hogy egy felhasználó más felhasználók kapcsolatait módosítsa.

```
// ellenorzi van-e mar baratsag/jeloles a ket user kozott
async function checkExistingFriendship(userId, targetId) {
  const smallId = Math.min(userId, targetId)
  const bigId = Math.max(userId, targetId)
  const sql = `
    SELECT id, status, requested_by
    FROM user_friendships
    WHERE user1_id = ? AND user2_id = ?
  `

  const [rows] = await pool.execute(sql, [smallId, bigId])
  return rows.length > 0 ? rows[0] : null
}

// barát jeloles kuldes
async function sendFriendRequest(userId, targetId) {
  const smallId = Math.min(userId, targetId)
  const bigId = Math.max(userId, targetId)
  const sql = `
    INSERT INTO user_friendships (user1_id, user2_id,
    requested_by, status)
    VALUES (?, ?, ?, 'pending')
  `

  const [result] = await pool.execute(sql, [smallId, bigId,
  userId])
  return result.insertId
}

// beerkezett (pending) jelolesek lekerese
async function getPendingRequests(userId) {
  const sql = `
    SELECT
      uf.id AS friendship_id,
      uf.requested_by,
      u.username AS requester_username,
      uf.created_at
    FROM user_friendships uf
    JOIN users u ON u.id = uf.requested_by
    WHERE (uf.user1_id = ? OR uf.user2_id = ?)
    AND uf.requested_by != ?
    AND uf.status = 'pending'
    ORDER BY uf.created_at DESC
  `

  const [rows] = await pool.execute(sql, [userId, userId, userId])
  return rows
}
```

2.31. ábra: barát jelölés függv

2.4.6. Receptek

A receptmodul a **meals.routes.js** fájlban található API-végpontokon keresztül kezeli a receptadatok **lekérését**, a felhasználói receptek **létrehozását**, **módosítását**, **törlését** és **megosztási kérelmét**. A végpontok csak bejelentkezett felhasználó számára érhetőek el, amit a **loginMw.requireAuthApi** middleware ellenőriz.

Főbb API-végpontok:

- **GET /data**
 - Feladata:
 - **Lekéri** a bejelentkezett felhasználó étrendhez szükséges adatait.
 - Visszaadja a felhasználói metrikákat, preferenciákat és az összes elérhető receptet.
 - Használt adatbázis-függvények:
 - **getUserMetrics**
 - **getUserPreferencesData**
 - **allFoods**
 - Visszatérési adat:
 - **metrics**
 - **preferences**
 - **foods**
- **POST /new**
 - Feladata:
 - Új saját recept **létrehozása** a bejelentkezett felhasználóhoz.
 - A beküldött receptadatok validálása.
 - A recept alapadatainak mentése.
 - A recepthez tartozó allergének külön rögzítése.
 - Middleware-ek:
 - **loginMw.requireAuthApi**
 - **foodsMw.validateNewFood**
 - Használt adatbázis-függvények:
 - **createUserFood**

- **insertFoodAllergen**
 - Fontos működés:
 - A **createUserFood** visszaadja az új recept azonosítóját.
 - Az allergének ehhez az azonosítóhoz kapcsolva kerülnek beszúrásra.
- **PUT /edit/:id**
 - Feladata:
 - Meglévő saját recept **módosítása**.
 - Ellenőrzi, hogy a recept az adott felhasználóhoz tartozik-e.
 - Frissíti a recept alapadatait.
 - Újragenerálja a recepthez tartozó allergénlistát.
 - Middleware-ek:
 - **loginMw.requireAuthApi**
 - **foodsMw.validateNewFood**
 - Használt adatbázis-függvények:
 - **updateUserFood**
 - **deleteFoodAllergens**
 - **insertFoodAllergen**
 - Fontos működés:
 - Ha az **updateUserFood** nem módosít sort, a rendszer **404-es** választ ad vissza.
 - Sikeres módosítás után a régi allergének törlődnek.
 - Az aktuális allergénlista újra bekerül az adatbázisba.
- **DELETE /delete/:id**
 - Feladata:
 - Saját recept **törlése**.
 - Csak a bejelentkezett felhasználóhoz tartozó recept törölhető.
 - Middleware:
 - **loginMw.requireAuthApi**
 - Használt adatbázis-függvény:
 - **deleteUserFood**

- Fontos működés:
 - Ha a recept nem található, vagy nem a felhasználóhoz tartozik, a rendszer **404**-es választ ad vissza.
- **POST /share/:id**
 - Feladata:
 - Megosztási kérelem **létrehozása** egy saját recepthoz.
 - A recept admin jóváhagyásra kerül, nem válik azonnal publikussá.
 - Middleware:
 - **loginMw.requireAuthApi**
 - Használt adatbázis-függvény:
 - **createShareTicket**
 - Fontos működés:
 - A rendszer a recept azonosítóját, a felhasználó azonosítóját és a recept nevét használja a kérelem létrehozásához.
 - A recept csak adminisztrátori jóváhagyás után kerülhet a publikus receptek közé.

Naplózás és hibakezelés:

- Sikeres műveletek esetén:
 - A rendszer a **log_id** függvénnyel rögzíti a fontosabb eseményeket.
 - Naplózott műveletek:
 - recept **létrehozása**
 - recept **módosítása**
 - recept **törlése**
 - **megosztási kérelem létrehozása**
- Hiba esetén:
 - A rendszer **500**-as választ küld vissza.
 - A hibák a **log_error** függvénnyel kerülnek rögzítésre.
 - A naplózáshoz a **requestIp.getClientIp(request)** alapján lekért **IP-cím** is felhasználásra kerül.

```

// log by id
async function log_id(id, action, desc, ip) {
  const userName = 'SELECT username FROM users WHERE id = ?';
  const insert = 'INSERT INTO logs(user_id, username, action,
description, ip_address) VALUES (?, ?, ?, ?, ?)';
  const [nameResult] = await pool.execute(userName, [id]);
  const [result] = await pool.execute(insert, [id, nameResult[0].
username, action, desc, ip]);
  return result.insertId;
}

// log by email
async function log_email(email, action, desc, ip) {
  const userName = 'SELECT id, username FROM users WHERE email = ?
';
  const insert = 'INSERT INTO logs(user_id, username, action,
description, ip_address) VALUES (?, ?, ?, ?, ?)';
  const [nameResult] = await pool.execute(userName, [email]);
  const [result] = await pool.execute(insert, [nameResult[0].id,
nameResult[0].username, action, desc, ip]);
  return result.insertId;
}

//log server failure
async function log_error(action, desc, ip) {
  const insert = 'INSERT INTO logs(action, description,type,
ip_address) VALUES (?, ?, ?, ?)';
  const [result] = await pool.execute(insert, [action, desc,
'error', ip]);
  return result.insertId;
}

```

2.32. ábra: log függvények

2.5. Tesztelés

2.5.1. Tesztelési környezet

A MuscleMind projekt tesztelése **Jest** alapú tesztkörnyezettel készült. A tesztek **két külön Jest projektként futnak**: külön backend és külön frontend tesztkörnyezettel.

Projekt	Tesztkörnyezet	Tesztfájlok helye
Backend	node	backend/tests/*.test.js
Frontend	jsdom	frontend/tests/*.test.js

A backend tesztekhez a projekt **backend/app.js** fájlja kerül importálásra. A **backend/server.js fájl** nem része a teszteknek, mert az indítaná el a valódi szerveret. Ezért a **Supertest** alapú **API tesztek** az Express alkalmazást szerverindítás nélkül tudják vizsgálni.

Tesztek futtatásához használt parancsok:

Parancs	Funkció
<code>npx jest</code>	Összes backend és frontend teszt futtatása
<code>npx jest --selectProjects backend</code>	Csak backend tesztek futtatása
<code>npx jest --selectProjects frontend</code>	Csak frontend tesztek futtatása
<code>npx jest --coverage</code>	Coverage riport készítése

Teszteléshez használt főbb csomagok:

Csomag	Felhasználás
<code>jest</code>	Tesztfuttatás, mockolás, assertionök
<code>supertest</code>	Backend API végpontok tesztelése
<code>jest-environment-jsdom</code>	Frontend DOM-alapú tesztek futtatása
<code>fetch mock</code>	Frontend API hívások helyettesítése tesztkörnyezetben

2.5.2 Tesztfájlok áttekintése

Tesztfájl	Tesztelt terület	Teszt típusa	Rövid cél
<code>backend/tests/basic.test.js</code>	Jest környezet	Sanity teszt	Ellenőrzi, hogy a Jest alapvetően működik

backend/tests/api404.test.js	backend/app.js	Supertest API teszt	API és nem API 404 válaszok ellenőrzése
backend/tests/authMiddleware.test.js	login.middleware.js	Unit teszt	Bejelentkez ési middleware -ek vizsgálata
backend/tests/adminMiddleware.test.js	isAdmin.middleware.js	Unit teszt	Admin jogosultság ellenőrzése
backend/tests/registrationMiddleware.test.js	registration.middleware.js	Unit teszt	Regisztráci ós validáció ellenőrzése
backend/tests/ticketMiddleware.test.js	ticket.middleware.js	Unit teszt	Ticket adatok validációja
backend/tests/workoutValidation.test.js	workout.middleware.js	Unit teszt	Edzésterv-v alidáció ellenőrzése
frontend/tests/api.test.js	frontend/javascript/api.js	Frontend API helper teszt	Fetch alapú API helper függvények ellenőrzése
frontend/tests/login.test.js	frontend/javascript/login.js	jsdom DOM teszt	Login űrlap kliensoldali működésén

			ek vizsgálata
frontend/tests/register.test.js	frontend/javascript/register.js	jsdom DOM teszt	Regisztrációs űrlap kliensoldali validációja

2.5.3. Backend tesztek

2.5.3.1. backend/tests/basic.test.js

Ez a **tesztfájl** egy alap környezetellenőrző tesztet tartalmaz. A teszt azt vizsgálja, hogy a **Jest** futtatókörnyezet működik-e.

Teszteset	Bemenet	Elvárt eredmény	Típus
Jest működik	1 + 1	Az eredmény 2	Pozitív

Ez a teszt nem üzleti logikát ellenőriz, hanem a **tesztkörnyezet működését igazolja**.

2.5.3.2. backend/tests/api404.test.js

Ez a fájl az Express alkalmazás 404-es **hibakezelését** vizsgálja **Supertest** segítségével. A teszt az app.js fájlt importálja, így nem indít valódi szerveret.

Teszteset	Bemenet	Elvárt eredmény	Típus
Nem létező API útvonal JSON 404 választ ad	GET /api/does-not-exist	HTTP 404, JSON válasz: Az API végpont nem található!	Negatív

Nem API útvonal HTML 404 oldalt ad	GET /does-not-exist-page	HTML válasz, amely tartalmazza az Error és error-code szövegeket	Negatív
---------------------------------------	-----------------------------	--	---------

A teszt igazolja, hogy az API útvonalak hibakezelése elkülönül a normál oldalak hibakezelésétől.

2.5.3.3. backend/tests/authMiddleware.test.js

Ez a fájl a bejelentkezéshez és jogosultsághoz kapcsolódó middleware-eket teszteli. Az adatbázisfüggvények mockolva vannak, így a teszt nem használ valódi adatbázist.

A tesztek a bejelentkezési folyamat backend oldali védelmét ellenőrzik.

Mock	Szerep
mockDb.email_exist	Email létezésének szimulálása

Teszteset	Bemenet / mock	Elvárt eredmény	Típus
validateLogin helyes adatokkal továbbenged	Helyes email és jelszó, email_exist = 1	next() meghívódik	Pozitív
validateLogin hibás adatokkal elutasít	Hibás email / jelszó	HTTP 400, Érvénytelen adat(ok)	Negatív
redirectIfLoggedIn bejelentkezett felhasználót átirányít	session.user.id = 42	res.redirect('/')	Negatív
redirectIfLoggedIn nem bejelentkezett felhasználót továbbenged	Üres session	next() meghívódik	Pozitív

requireAuthPage anonim felhasználót tilt	Üres session	401-es oldal küldése	Negatív
requireAuthPage bejelentkezett felhasználót továbbenged	session.user.id = 10	next() meghívódik	Pozitív

2.5.3.4. backend/tests/adminMiddleware.test.js

Ez a fájl az admin jogosultság ellenőrzéséért felelős middleware-t teszteli. A tesztek igazolják, hogy az admin felülethez csak admin jogosultságú felhasználó férhet hozzá.

Teszteset	Bemenet / mock	Elvárt eredmény	Típus
Admin felhasználó továbbengedése	session.user.admin = 1	next() meghívódik	Pozitív
Nem admin felhasználó tiltása	session.user.admin = 0	403-as oldal küldése	Negatív
Session nélküli kérés hibakezelése	Üres request objektum	next(error) meghívódik	Negatív

2.5.3.5. backend/tests/registrationMiddleware.test.js

Ez a fájl a regisztrációs adatok backend oldali validációját teszteli. Az adatbázis-lekérdezések mockoltak. A tesztek azt ellenőrzik, hogy a regisztráció csak megfelelően kitöltött adatokkal folytatódhat.

Mock	Szerep
mockDb.username_exist	Felhasználónév létezésének szimulálása
mockDb.email_exist	Email létezésének szimulálása

Teszteset	Bemenet / mock	Elvárt eredmény	Típus
-----------	----------------	-----------------	-------

Helyes regisztrációs adatok elfogadása	Teljes, helyes regisztrációs adat	next() meghívódik	Pozitív
Hiányzó kötelező mező elutasítása	Hiányzó email mező	HTTP 400, Hiányzó vagy érvénytelen mezők	Negatív

2.5.3.6. backend/tests/ticketMiddleware.test.js

Ez a fájl a ticket létrehozás validációját teszteli. Az adatbázisfüggvények mockoltak, így a teszt nem hoz létre valódi ticketet. A teszt biztosítja, hogy a ticket rendszer ne fogadjon el üres vagy érvénytelen adatokat.

Mock	Szerep
mockDb.limitTicketCreation	Ticket létrehozási limit szimulálása
mockDb.findTicketEmail	Felhasználói email keresés szimulálása
mockDb.findPreId	Kapcsolódó rekord azonosítójának szimulálása

Teszteset	Bemenet / mock	Elvárt eredmény	Típus
Helyes ticket adatok elfogadása	Helyes ticket adat, findPreId = [1]	next() meghívódik	Pozitív
Üres üzenet elutasítása	message = ''	HTTP 400, Hibás adatok!	Negatív
Hibás kategória elutasítása	category = 'invalid'	HTTP 400, Hibás adatok!	Negatív

2.5.3.7. backend/tests/workoutValidation.test.js

Ez a fájl az edzésterv létrehozásához kapcsolódó validációkat vizsgálja. A teszt az edzésterv alapvető szerkezeti szabályait ellenőrzi.

Mock	Szerep
mockDb.exerciseExist	Gyakorlat létezésének szimulálása

Teszteset	Bemenet / mock	Elvárt eredmény	Típus
Helyes edzésterv elfogadása	Érvényes tervadatok, exerciseExist = true	next() meghívódik	Pozitív
Duplikált napok elutasítása	Két azonos dayNumber	HTTP 400, Napok adatainak feldolgozása sikertelen!	Negatív
Duplikált gyakorlatok elutasítása	Egy napon belül két azonos exerciseId	HTTP 400, Duplikáció a gyakorlatok között!	Negatív
Hiányos edzésterv elutasítása	Üres név, üres naplista	HTTP 400, Adj nevet az edzéstervnek.	Negatív

2.5.4. Frontend tesztek

2.5.4.1. frontend/tests/api.test.js

Ez a fájl a frontend api.js helper függvényeit teszteli. A valódi hálózati kérések helyett a fetch függvény mockolva van. A teszt azt ellenőrzi, hogy a központi frontend API helper réteg helyesen kezeli az alapvető HTTP műveleteket.

A frontend api.js ES module szintaxist használ, ezért a teszt a fájlt izoláltan tölti be. A tesztkörnyezet a fetch hívásokat nem küldi el valódi backend felé.

Teszteset	Bemenet / mock	Elvárt eredmény	Típus
getFetch GET kérést küld	Sikeres mockolt fetch válasz	GET hívás helyes fejléccel, JSON válasz feldolgozása	Pozitív
postNewPlan POST kérést küld JSON body-val	Sikeres mockolt fetch válasz	POST hívás JSON body-val	Pozitív
deleteFetch hibás válasznál kivételt dob	fetch.ok = false	Delete failed hiba dobása	Negatív

2.5.4.2. frontend/tests/login.test.js

Ez a fájl a login oldal kliensoldali működését teszteli jsdom környezetben. A valódi API hívás helyett a login függvény mockolva van. A teszt a login űrlap eseménykezelését, a gomb aktiválását, a sikeres és hibás válaszok kezelését ellenőrzi.

DOM elem	Szerep
login-form	Bejelentkezési űrlap
email-login	Email mező
password-login	Jelszó mező
login-in	Bejelentkezés gomb
password-feedback	Visszajelző üzenet

Teszteset	Bemenet / mock	Elvárt eredmény	Típus
Helyes login adatok feldolgozása	Helyes email és jelszó, sikeres mockolt login válasz	Login helper meghívódik, sikerüzenet megjelenik, sessionStorage érték beáll	Pozitív

Hibás login válasz kezelése	Mockolt login hiba: Hibás adat	Hibaüzenet megjelenik, félkövér piros visszajelzés	Negatív
--------------------------------	-----------------------------------	---	---------

2.5.4.3. frontend/tests/register.test.js

Ez a fájl a regisztrációs oldal kliensoldali validációját teszteli jsdom környezetben. A valódi backend hívás helyett a registration függvény mockolva van. A teszt a regisztrációs űrlap legfontosabb kliensoldali ellenőrzéseit vizsgálja.

DOM elem	Szerep
registration-form	Regisztrációs űrlap
last-name	Vezetéknév mező
first-name	Keresztnév mező
userName	Felhasználónév mező
email-regist	Email mező
password-regist	Jelszó mező
password-confirm-regist	Jelszó megerősítése mező
regist-in	Regisztráció gomb
password-feedback	Visszajelző üzenet

Teszteset	Bemenet / mock	Elvárt eredmény	Típus
Hiányzó mezők kezelése	Hiányos űrlap	Hibaüzenet jelenik meg, registration helper nem hívódik meg	Negatív

Nem egyező jelszavak kezelése	Két különböző jelszó	A gomb tiltva marad, mismatch üzenet megjelenik	Negatív
Helyes regisztrációs adatok kezelése	Teljes, helyes űrlapadatok, sikeres mockolt válasz	Registration helper meghívódik, sikerüzenet megjelenik	Pozitív

2.5.5. Pozitív és negatív tesztesetek aránya

A tesztcsomag pozitív és negatív eseteket is tartalmaz. A pozitív tesztek azt ellenőrzik, hogy helyes adatok esetén a folyamat továbbhalad. A negatív tesztek azt vizsgálják, hogy hibás vagy hiányos adatok esetén a rendszer megfelelően tilt, hibát jelez vagy nem engedi tovább a folyamatot.

Ez által nemcsak a működő eseteket ellenőrzi, hanem azt is, hogy a rendszer megfelelően kezeli-e a hibás bemeneteket.

2.5.6. Teszteredmények

```
oliver@Mac MuscleMind % npx jest --coverage
PASS backend backend/tests/api404.test.js
PASS frontend frontend/tests/register.test.js
PASS backend backend/tests/adminMiddleware.test.js
PASS frontend frontend/tests/login.test.js
PASS frontend frontend/tests/api.test.js
PASS backend backend/tests/ticketMiddleware.test.js
PASS backend backend/tests/workoutValidation.test.js
PASS backend backend/tests/registrationMiddleware.test.js
PASS backend backend/tests/authMiddleware.test.js
PASS backend backend/tests/basic.test.js
```

2.2. ábra: Jest tesztfuttatás eredménye

```
Test Suites: 10 passed, 10 total
Tests: 30 passed, 30 total
Snapshots: 0 total
Time: 1.208 s
```

2.3. ábra: Sikeres Jest tesztfuttatás eredménye

2.5.7. Coverage eredmények

A coverage riport a teljes projektre számolva az alábbi eredményt adta:

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	15.04	11.87	4.36	15.01	
backend	63.93	7.14	15.78	63.93	
app.js	63.93	7.14	15.78	63.93	...57,61,65,69,73,77,81,85,89,93,97,101,124-140
backend/api	15.91	0	0	16.07	...
admin.routes.js	9.45	0	0	9.45	...0-555,561-587,594-609,615-641,647-674,680-712
api.js	93.1	100	0	93.1	21-24
auth.routes.js	14.55	0	0	14.55	...5-119,126-139,146-282,288-324,330-368,375-411
meals.routes.js	15.73	0	0	15.73	12-32,40-69,77-115,123-154,162-188

2.4. ábra: Coverage riport eredménye

Az **összesített coverage** érték azért alacsony, mert a riport a **teljes backend** mappát figyelembe veszi, beleértve a nagy méretű database.js fájlt és az összes API route-ot is. Ezeket a tesztek nem fedik le teljes mértékben, mivel a **tesztelés elsődleges célja a főbb middleware-ek, validációk, API hibakezelés és frontend űrlaplogikák ellenőrzése volt.**

A **frontend tesztek sikeresen lefutottak**, azonban a coverage HTML riport elsősorban backend fájlokat tartalmaz. Ennek oka, hogy a frontend fájlokat a tesztek nem a megszokott módon töltik be, ezért ezeknél a coverage eredmény nem mutat teljesen pontos képet.

2.5.8. Összegzés

A **MuscleMind** projektben összesen **30 Jest alapú teszt** készült. A tesztek **külön kezelik a backend és a frontend részt**. A backend tesztek főként a **middleware-eket, a jogosultságkezelést, a validációkat és az API hibakezelését** ellenőrzik. A **Supertest** segítségével az **API 404-es** válaszait úgy tudtuk tesztelni, hogy nem kellett külön elindítani a szerveret.

A frontend tesztek **jsdom** környezetben futnak. Ezek a **login, a regisztráció és az API helper függvények** működését vizsgálják. A tesztek nem küldenek valódi hálózati kéréseket, és nem módosítanak adatbázist, hanem **mockolt függvényekkel dolgoznak**. Így a tesztek **gyorsan és biztonságosan futtathatók**.

A **coverage riport** alapján a legjobban lefedett részek az admin jogosultságkezelés, a regisztrációs validáció és a ticket validáció. A tesztelés célja nem az volt, hogy a teljes rendszert teljesen lefedje, hanem hogy a **legfontosabb validációs, jogosultsági és hibakezelési részek ellenőrizve legyenek**.

2.6. Továbbfejlesztési lehetőségek

2.6.1. Pontosabb muscle group XP számítás aktivitási szorzókkal

A MuscleMind egyik fontosabb továbbfejlesztési lehetősége a **muscle group XP számítás pontosítása**. A jelenlegi működés bővíthető lenne úgy, hogy egy

gyakorlat volumene ne csak egyetlen izomcsoporthoz kapcsolódjon, hanem az adott gyakorlat által érintett **összes releváns izomcsoport**hoz.

Ehhez minden gyakorlathoz tartozhatna egy **aktivitási szorzó**, amely megadja, hogy az adott izomcsoport milyen mértékben vesz részt a mozdulatban. Például fekvőnyomás esetén a mellizom nagyobb, míg a váll és a tricepsz kisebb arányban kapna XP-t. Ez pontosabb **fejlődéskövetést** és reálisabb izomcsoport-alapú statisztikákat eredményezne.

2.6.2. Adaptív edzésterv-ajánló rendszer

A későbbi fejlesztések során bevezethető lenne egy **adaptív edzésterv-ajánló rendszer**, amely a felhasználó céljai, profiladatai és korábbi edzésaktivitása alapján javasolna edzésterveket.

A rendszer a meglévő célalapú szűrésre építve később **adaptív edzésterv-ajánlóval** is bővíthető lenne. Ez a felhasználó profiladatai és korábbi edzésaktivitása alapján segítené a relevánsabb tervek előtérbe helyezését, miközben az összes edzésterv továbbra is elérhető maradna.

2.6.3. Profilkép-feltöltés és recept komment szekció

A felhasználói élmény és a közösségi funkciók erősítése érdekében a rendszer bővíthető lenne **profilkép-feltöltési lehetőséggel**, valamint a receptekhez kapcsolódó **komment szekcióval**.

A profilkép személyesebbé tenné a felhasználói fiókot, míg a kommentelési lehetőség teret adna a receptekkel kapcsolatos véleményeknek, tapasztalatoknak és javaslatoknak. Ezáltal a receptkezelés nemcsak keresési funkcióként, hanem interaktívabb közösségi elemként is működhetne.

2.6.4. Gamifikációs és motivációs rendszer bevezetése

A felhasználói aktivitás növelése érdekében a rendszer bővíthető lenne **gamifikációs elemekkel**. Ilyen funkció lehetne például a jelvények, heti kihívások vagy teljesítményalapú mérföldkövek bevezetése.

Ezek az elemek nem az alkalmazás alpműködését módosítanák, hanem a **hosszabb távú felhasználói elköteleződést** támogatnák. A cél az lenne, hogy a felhasználók motiváltabbak legyenek az edzéseik rendszeres naplózásában és céljaik követésében.

2.6.5. AI személyes tanácsadó chatbot implementálása

Egy további fejlesztési irány lehetne egy **AI alapú személyes tanácsadó chatbot** beépítése. A chatbot a felhasználó által megadott adatok, célok és kérdések alapján tudna edzéssel, étkezéssel vagy fejlődéssel kapcsolatos tanácsokat adni.

Ez a funkció segíthetne például az **edzésterv-választásban**, étrendi ötletek megfogalmazásában vagy a fejlődési adatok értelmezésében. A chatbot személyesebb támogatást biztosítana, és interaktívabbá tenné a MuscleMind használatát.

2.7. Összegzés

A MuscleMind jelenlegi verziója **moduláris backendből és statikus frontendből** épül fel. A backend **REST API végpontokra, elkülönített middleware rétegre, MySQL adatbázisra és külön kezelt üzleti logikára** támaszkodik. Ez a felépítés átláthatóvá teszi a főbb funkciók, például a **profilkezelés, edzéstervezés, receptkeresés, jegyrendszer és adminisztráció** működését.

A fejlesztés során fontos szempont volt, hogy az **adatkezelés, jogosultságkezelés, validáció és tranzakciókezelés** külön rétegekben, követhető módon jelenjen meg. Ez megkönnyíti a **hibakeresést, a meglévő funkciók módosítását és az új modulok későbbi beépítését**.

A dokumentációban leírt továbbfejlesztési lehetőségek alapján a rendszer **több irányban is bővíthető**. Ezek a fejlesztések elsősorban a **felhasználói élmény továbbnövelését és a rendszer intelligensebb működését** támogatnák. A projekt jelenlegi állapota **stabil alapot ad a további fejlesztésekhez**, mivel a főbb funkciók **elkülönített szerkezetben, adatbázis-kapcsolattal és jogosultságkezeléssel** működnek.

3. Felhasználói dokumentáció

3.1. A program célja és főbb funkciói

3.1.1. A program célja

A MuscleMind egy webalapú fitneszkövető alkalmazás, amelynek célja, hogy támogassa a felhasználókat személyes fitneszcéljaik elérésében. Az alkalmazás személyre szabott edzésterveket, étrendjavaslatokat, edzésnaplózási lehetőséget, valamint részletes statisztikai kimutatásokat biztosít.

A rendszer célja, hogy egyetlen felületen tegye elérhetővé az edzéshez, táplálkozáshoz és fejlődéskövetéshez kapcsolódó legfontosabb funkciókat. Emellett közösségi elemeket is tartalmaz, például ranglistát, barátkezelést és közösségi receptek megosztását.

3.1.2. Főbb funkciók

Felhasználókezelés:

Az alkalmazás lehetőséget biztosít felhasználói fiók létrehozására, bejelentkezésre, személyes profil kezelésére, valamint jelszó-ementetető és jelszó-visszaállítás használatára. A jelszavak biztonságos, hash-elt formában kerülnek tárolásra. A bejelentkezett állapotot a rendszer munkamenet-alapú hitelesítéssel kezeli.

Személyre szabott kérdőív:

A regisztrációt követően a felhasználó egy részletes kérdőívet tölt ki. A kérdőív célja, hogy felmérje a felhasználó edzési szintjét, céljait, étrendi

szokásait és egyéb preferenciáit. Az itt megadott adatok alapján az alkalmazás személyre szabottabb edzés és étrendjavaslatokat tud biztosítani.

Edzéstervek kezelése:

A rendszer előre elkészített edzésterveket kínál kezdő, középhaladó és haladó felhasználók számára. Emellett lehetőséget ad saját edzéstervek létrehozására, testreszabására, aktiválására és törlésére. A felhasználó nyomon követheti az edzések teljesítését és az egyes gyakorlatokhoz tartozó adatokat.

Étrendkezelés:

Az alkalmazás receptadatbázist tartalmaz, amelyben a receptek tápértékadatokkal együtt jelennek meg. A felhasználó szűrhet cél, étrendtípus, allergének, nehézségi szint és tápérték alapján. Saját receptek is létrehozhatók és megoszthatók a közösséggel.

Naptár és edzésnapló:

A naptárfunkció segítségével a felhasználó áttekintheti, megtervezheti és követheti edzéseit. Az edzések státusza megkülönböztethető, például függőben lévő, elkezdett, befejezett, kihagyott vagy pihenőnapként jelölt edzésként. Az edzésnaplóban rögzíthetők a sorozatok, ismétlések és használt súlyok.

Napi edzés emlékeztetés:

Az edzés emlékeztető funkció értesítésekkel segíti a rendszeres edzésrutin betartását, így a felhasználó sosem feledkezik meg a betervezett edzésekről. Minden bejelentkezésnél egy kis felugró ablak formájában jelenik meg a felhasználónak a naptár segítségével követett aznapi edzés részletes leírása.

Statisztikák és elemzések:

A statisztikai modul részletes képet ad a felhasználó fejlődéséről. Megjeleníti többek között a teljesített edzések számát, az összesített emelt súlyt, az XP pontokat, az izomcsoportonkénti terhelést, valamint a személyes rekordokat. A grafikus megjelenítés Chart.js segítségével történik.

Közösségi funkciók:

A MuscleMind közösségi funkciói közé tartozik a ranglista, a pontrendszer, a barátok kezelése, valamint a közösségi receptek megosztása. Ezek a funkciók motivációs szerepet töltenek be, és támogatják a felhasználók közötti kapcsolatépítést.

Adminisztrációs felület

Az adminisztrációs felület kizárólag adminisztrátori jogosultsággal érhető el. Itt lehetőség van a felhasználók kezelésére, a tartalmak moderálására, a hibajegyek kezelésére, valamint a rendszerrel kapcsolatos statisztikák és naplók megtekintésére.

3.2. Szükséges hardver- és szoftverkörnyezet

3.2.1. Hardverkövetelmények

Az alkalmazás webes alapú, ezért a használatához nincs szükség speciális hardverre. A megfelelő működéshez az alábbi minimális hardverkörnyezet ajánlott:

- modern processzor, például Intel Core i3 vagy AMD Ryzen 3, illetve ezekkel egyenértékű CPU
- legalább 4 GB RAM, ajánlott 8 GB vagy több
- legalább 2 GB szabad tárhely
- stabil internetkapcsolat

3.2.2. Szoftver Követelmények

Fejlesztői környezet:

Amennyiben az alkalmazást helyi környezetben szeretnénk futtatni vagy fejleszteni, az alábbi szoftverek szükségesek:

- Node.js 18.x vagy újabb verzió;
- npm 8.x vagy újabb verzió, illetve alternatívaként Yarn;

- MySQL 8.0 vagy újabb verzió;
- modern webböngésző.

Felhasználói környezet:

A végfelhasználók számára az alkalmazás használatához elegendő egy modern webböngésző. A támogatott környezetek:

- Windows 10 vagy Windows 11;
- macOS 10.15 vagy újabb verzió;
- Linux disztribúciók;
- Google Chrome 90 vagy újabb;
- Mozilla Firefox 88 vagy újabb;
- Microsoft Edge 90 vagy újabb;
- Safari 14 vagy újabb.

Adatbázis-követelmények:

Az alkalmazás MySQL adatbázist használ. Az adatbázis ajánlott karakterkódolása UTF-8, magyar rendezési szabállyal.

Egyéb követelmények:

Az e-mail-küldési funkció használatához Brevo API-kulcs szükséges. Ez fejlesztői környezetben opcionális, azonban a jelszó- emlékeztető és egyéb e-mail-alapú funkciók teljes körű működéséhez szükség van rá.

3.3. Telepítés és indítás

3.3.1. Előfeltételek

A telepítés megkezdése előtt ellenőrizni kell, hogy a számítógépen telepítve van-e a Node.js és a MySQL szerver.

=====

\$npm install

=====

3.3.2. A projekt letöltése

A projekt Git használatával tölthető le:

=====

bash

git clone <<https://github.com/sz-ol1ver/MuscleMind.git>>

cd MuscleMind

=====

Amennyiben a projekt tömörített fájlban áll rendelkezésre, a fájlt ki kell csomagolni, majd a projekt gyökérkönyvtárába kell navigálni.

3.3.3. Backend telepítése

A backend függőségeinek telepítéséhez a következő parancsokat kell futtatni:

=====

bash

```
cd MuscleMind/backend
```

```
npm install
```

```
=====
```

3.3.4. Adatbázis beállítása

Az adatbázis beállításának lépései:

1. MySQL-ben létre kell hozni egy `musclemind` nevű adatbázist.
2. Importálni kell az adatbázissémát a `backend/sql/database.sql` fájlból.
3. Igény esetén importálhatók a tesztadatok a `backend/sql/db\_quick.sql` fájlból.

3.3.5. Környezeti változók beállítása

A környezeti változók beállításához a `backend/.env.example` fájlt át kell másolni `backend/.env` néven.

Ezt követően megadható a Brevo API-kulcs:

```
=====
env
BREVO_API_KEY=your_actual_api_key_here
=====
```

3.3.6. A szerver indítása

Fejlesztői módban a szerver az alábbi paranccsal indítható:

```
=====
bash
npm run dev
=====
```

Éles módban a következő parancs használható:

```
=====
bash
npm run start
=====
```

3.3.7. Az alkalmazás elérése

Sikeres indítás után az alkalmazás a következő címen érhető el böngészőből:

```
=====
text
http://localhost:3000
=====
```

3.3.8. Lehetséges telepítési problémák

PowerShell végrehajtási házirend:

Windows rendszeren előfordulhat, hogy a PowerShell nem engedélyezi a szkriptek futtatását. Ebben az esetben az alábbi hibaüzenet jelenhet meg:

```
=====
text
Cannot be loaded because running scripts is disabled on this system.
=====
```

A probléma megoldásához rendszergazdai jogosultsággal meg kell nyitni a PowerShellt, majd futtatni kell a következő parancsot:

```
=====
powershell
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
=====
```

(A parancs futtatása után célszerű újraindítani a fejlesztőkörnyezetet.)

Foglalt port:

Amennyiben a 3000-es port már használatban van, a következő paranccsal felszabadítható:

```
=====
bash
npx kill-port 3000
=====
```

3.4. A program részletes bemutatása

3.4.1. Regisztráció és bejelentkezés

Regisztráció:

A regisztráció a bejelentkezési oldalon található „Nincs még fiókod? Regisztrálj” hivatkozáson keresztül érhető el. A regisztrációs űrlapon a felhasználónak meg kell adnia a vezetéknevét, keresztnévét, felhasználónevét, e-mail-címét és jelszavát.

A felhasználónévnek és az e-mail-címnek egyedinek kell lennie. A jelszónak legalább 8 karakterből kell állnia. Sikeres regisztráció után a

rendszer automatikusan bejelentkezteti a felhasználót, majd átirányítja a kérdőív oldalára.



MuscleMind
Regisztráció

Vezetéknév

Keresztnév

Felhasználónév

E-mail cím

Jelszó

Jelszó meg erősítése

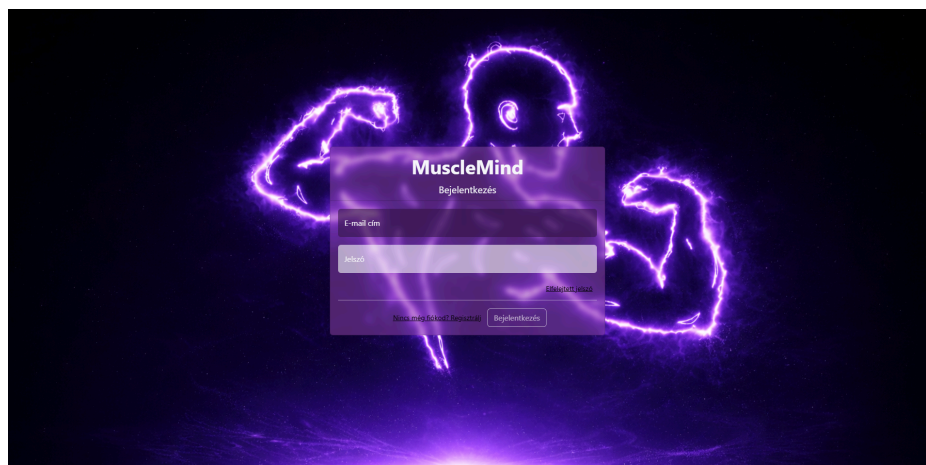
[Van már MuscleMind fiókja? be](#)

3.1. Regisztráció

Bejelentkezés:

Bejelentkezéskor a felhasználónak az e-mail-címét és jelszavát kell megadnia. Elfelejtett jelszó esetén a rendszer jelszó-visszaállító funkciót biztosít.

A bejelentkezett állapotot a rendszer munkamenettel kezeli. A munkamenet 30 perc inaktivitás után automatikusan lejár, amely biztonsági szempontból védi a felhasználói fiókot.



3.2. Bejelentkezés

3.4.2. Kérdőív kitöltése

A regisztráció után a felhasználónak ki kell töltenie egy kérdőívet. A kérdőív célja, hogy az alkalmazás pontosabb képet kapjon a felhasználó fizikai adatairól, céljairól és edzési szokásairól.

A kérdőív főbb témakörei:

- személyes adatok, például születési dátum és magasság
- fitneszcél, például tömegnövelés, szálkásítás vagy szinttartás
- edzési szint, például kezdő, középhaladó vagy haladó

- heti edzések száma
- edzés helyszíne, például konditerem vagy otthoni edzés
- étrendtípus, például mindenevő, vegetáriánus vagy vegán
- napi étkezések száma

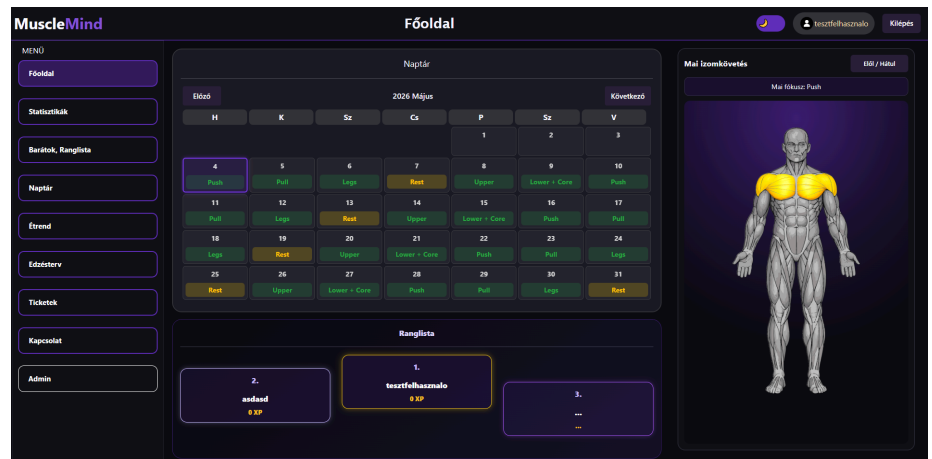
A kérdőív kitöltését követően a felhasználó a főoldalra kerül, ahol az alkalmazás további funkciói elérhetővé válnak.

3.4.3. Főoldal és irányítópult

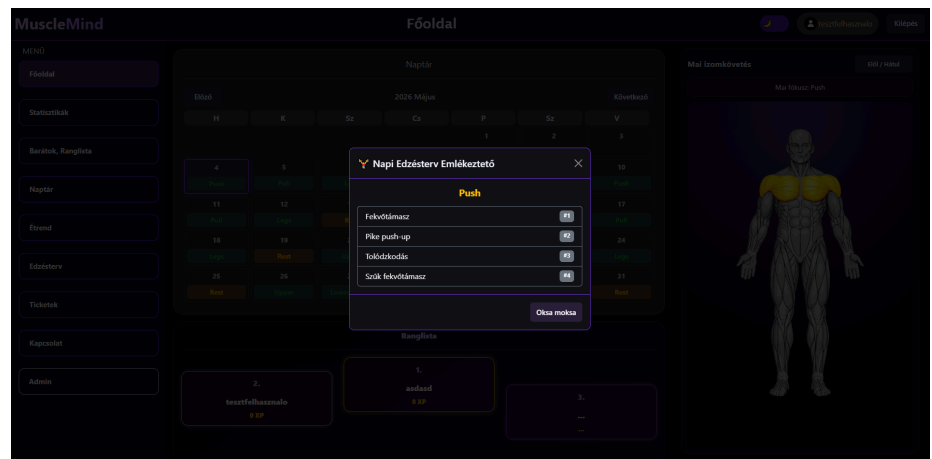
A főoldal a projekt központi felülete, ahol a felhasználó áttekintést kap az aktuális edzéseiről(naptár), ranglistán való helyezéséről, aznapi érintett izmokról(izomkövető).

A főoldal főbb elemei:

- naptármodul az aktuális hónap edzéseinek áttekintésére
- ranglista a legjobb felhasználók XP pontjai alapján
- napi izomkövető megjelenítés
- navigációs menü az alkalmazás fő funkcióinak eléréséhez
- edzés emlékeztető



3.3. Főoldal



3.4 Edzés emlékeztető

3.4.4. Edzéstervek kezelése

Az edzéstervek menüpont a webalkalmazás egyik központi funkciója. Itt a felhasználó megtekintheti saját edzésterveit, kiválaszthatja az ajánlott edzésterveket, illetve új tervet is létrehozhat.

Saját edzéstervek:

A saját edzéstervek listájában megjelennek a felhasználó által létrehozott edzéstervek. Ezek szerkeszthetők, törölhetők vagy aktív edzéstervként beállíthatók.

Ajánlott edzéstervek:

Az ajánlott edzéstervek a rendszer által előre elkészített tervek. Ezek különböző szempontok alapján szűrhetők:

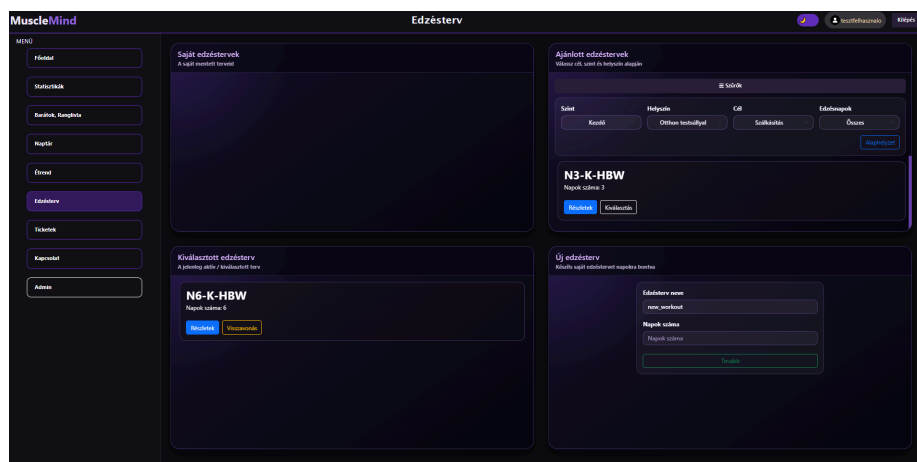
- edzési szint
- edzés helyszíne
- fitneszcél
- heti edzésnapok száma

Aktív edzésterv:

Az aktív edzésterv az a terv, amely alapján a rendszer a felhasználó aktuális edzéseit kezeli. Az edzésterv napokra bontva jelenik meg, a napokon belül pedig a gyakorlatok listája látható.

Új edzésterv létrehozása:

Új edzésterv létrehozásakor a felhasználó megadhatja a terv nevét, a napok számát, az egyes napokhoz tartozó gyakorlatokat,



valamint a pihenőnapokat.

3.5. Edzésterv létrehozás, ajánlott edzéstervek, szűrés

3.4.5. Étrend és receptek kezelése

Az étrend menüpont a táplálkozással kapcsolatos funkciókat tartalmazza. A felhasználó recepteket kereshet, szűrhet, megtekinthet, létrehozhat és megoszthat más felhasználókkal.

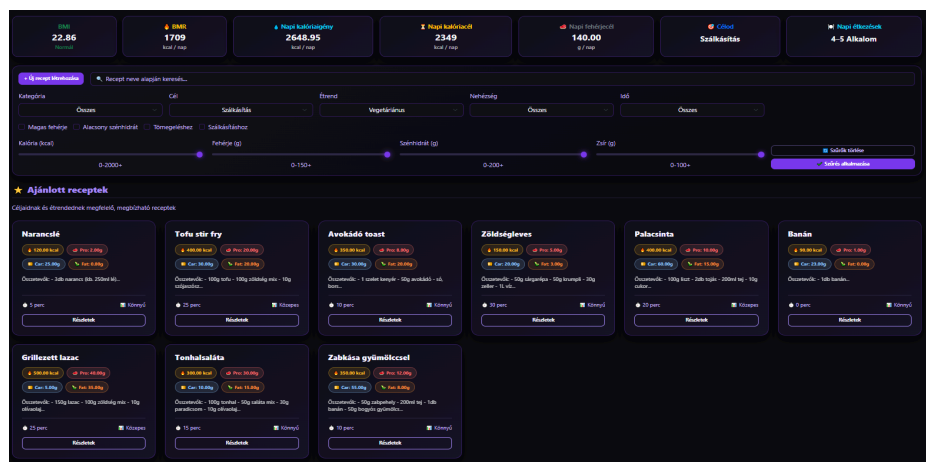
A receptkeresés során több szűrési lehetőség áll rendelkezésre:

- név szerinti keresés
- kategória szerinti szűrés, például reggeli, ebéd, vacsora, snack, desszert vagy ital
- cél szerinti szűrés
- étrendtípus szerinti szűrés
- nehézségi szint szerinti szűrés
- elkészítési idő szerinti szűrés
- tápérték szerinti szűrés

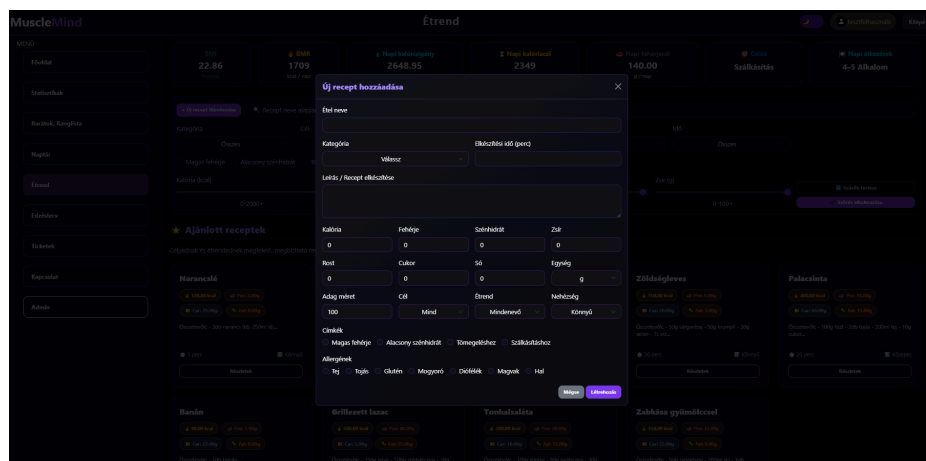
A receptek három fő csoportba sorolhatók:

1. ajánlott receptek
2. közösségi receptek
3. saját receptek

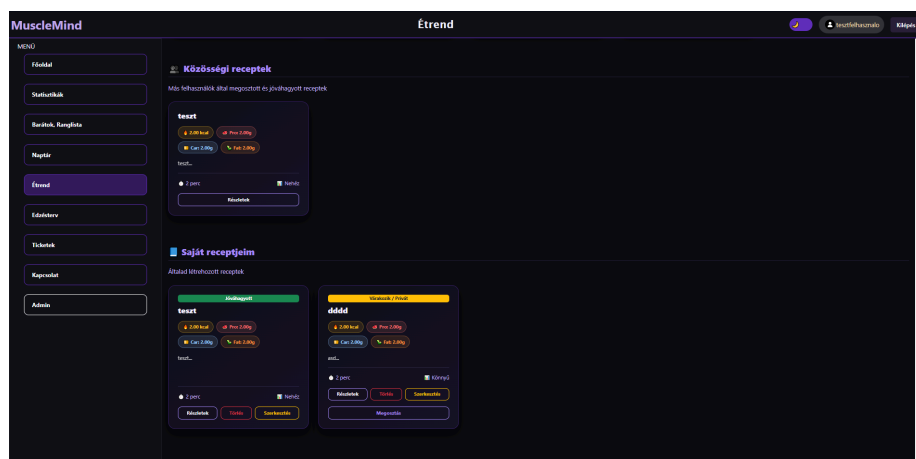
A recept részleteinél megjelennek a tápértékadatok, az elkészítési leírás, az allergének, a kategóriák, a címkék, valamint az adagméret. A tápértékadatok közé tartozik a kalória, fehérje, szénhidrát, zsír, rost, cukor és só mennyisége.



3.6. Étrend Kezelő oldal felépítése, szűrő



3.7. Étrend Kezelő oldal új recept létrehozása



3.8 Étrend Kezelő oldal közösségi receptek, saját receptek

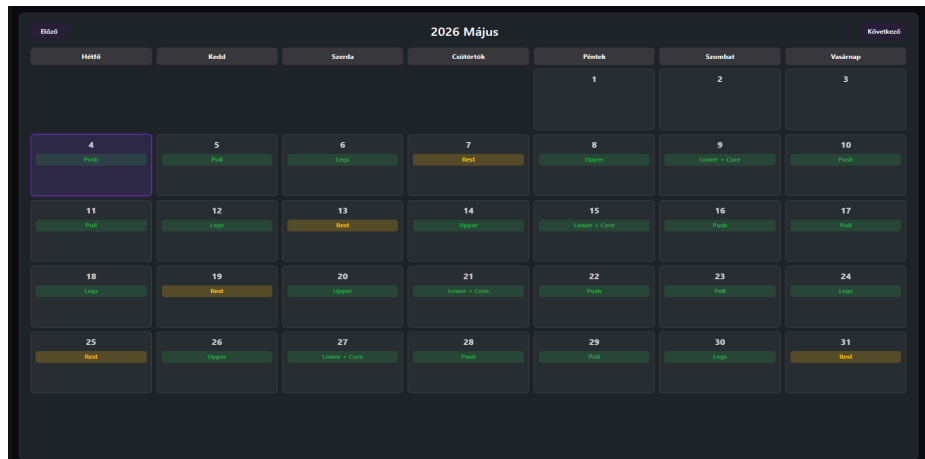
3.4.6. Naptár és edzésnapló

A naptár menüpont az edzések tervezésére és követésére szolgál. A felhasználó havi nézetben láthatja az edzésnapokat, valamint megtekintheti az egyes napokhoz tartozó edzésrészleteket.

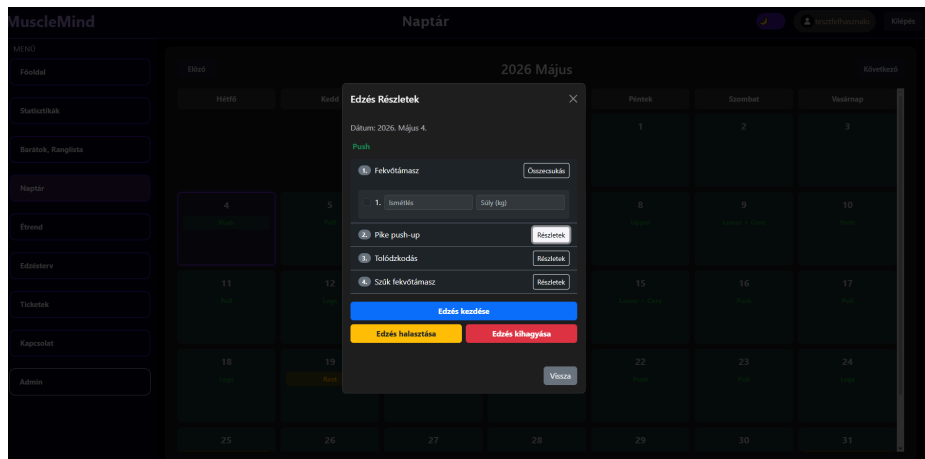
Az edzésekhez az alábbi státuszok tartozhatnak:

- függőben lévő edzés
- elkezdett edzés
- befejezett edzés
- kihagyott edzés
- pihenőnap

Az edzésnaplóban a felhasználó rögzítheti az egyes gyakorlatokhoz tartozó sorozatokat, ismétlésszámokat és használt súlyokat (saját testsúlyos edzéseknél automatikusan a felhasználó által megadott testsúllyal számol). Az edzés befejezése után az adatok bekerülnek a statisztikai modulba.



3.9 Naptár oldal edzésekkel feltöltve



3.10 Edzés kezelés(halasztás, kihagyás), sorozatok, ismétlés, súly hozzáadása

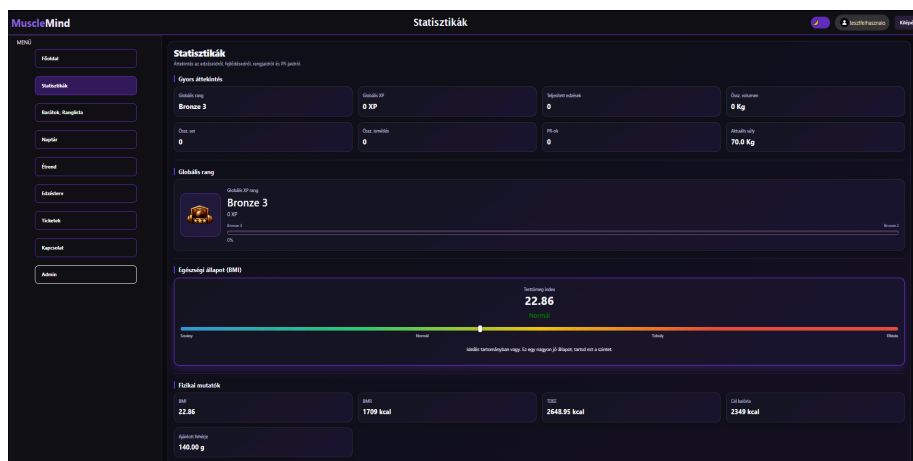
3.4.7. Statisztikák

A statisztikák menüpont részletes áttekintést nyújt a felhasználó edzési előrehaladásáról.

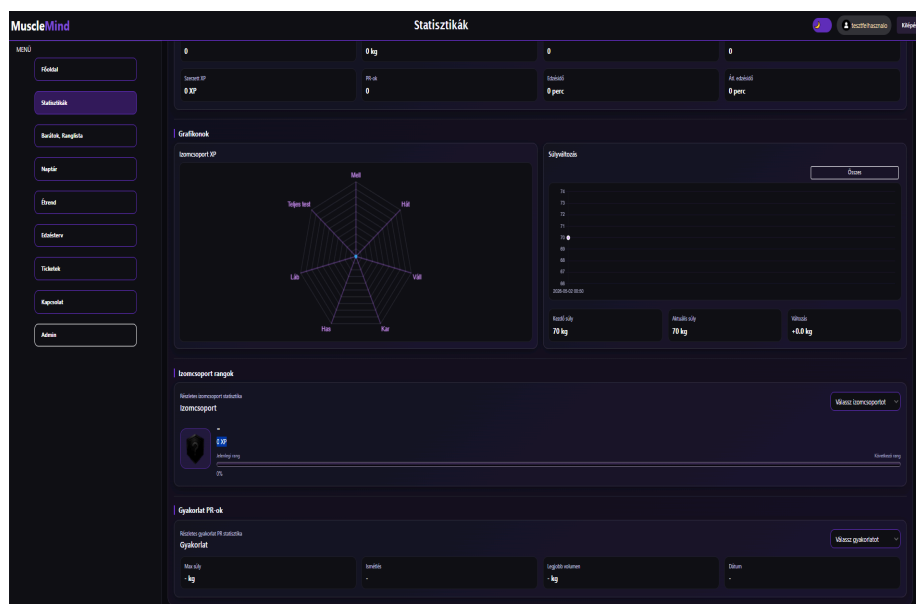
A rendszer többféle adatot jelenít meg, például:

- teljesített edzések száma
- összes emelt súly
- összes sorozat és ismétlés
- összes edzésidő
- XP pontok
- izomcsoportonkénti fejlődés
- személyes rekordok

Az adatok napi, heti és havi bontásban is megtekinthetők. A grafikonok interaktív formában jelennek meg, amiket a Chart.js könyvtár biztosít.



3.11 Statisztika oldal (1)



3.12 Statisztika oldal (2.) Grafikonos felület

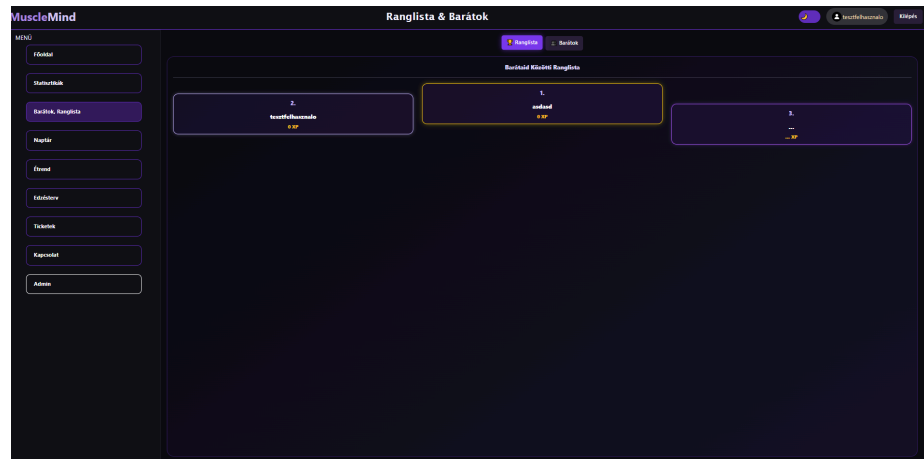
3.4.8. Ranglista és közösségi funkciók

A ranglista menüpont a közösségi funkciók egyik fő eleme. A felhasználók XP pontjaik alapján kerülnek rangsorolásra. A ranglista időszak szerint is szűrhető, például heti, havi vagy éves bontásban.

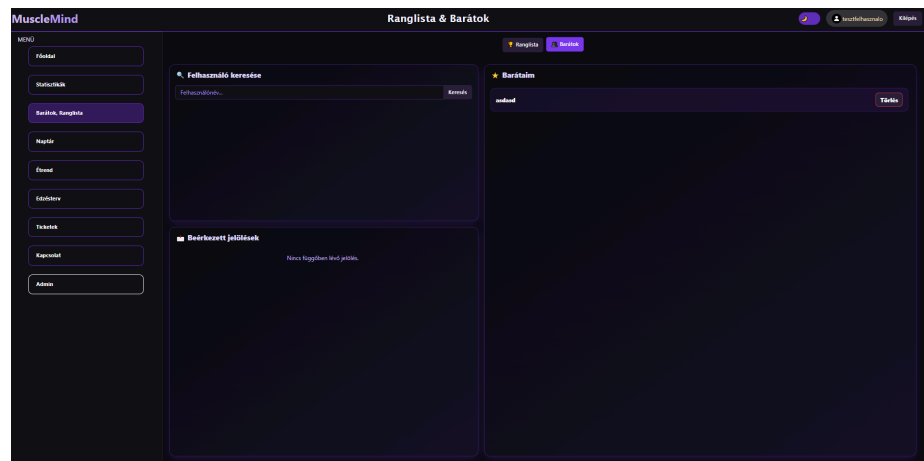
A közösségi funkciók közé tartozik:

- barátkérések küldése
- beérkezett barátkérések elfogadása vagy elutasítása
- barátok listájának kezelése
- közösségi receptek megtekintése, megosztása

Ezek a funkciók növelik a felhasználói aktivitást, és motivációt biztosítanak a rendszer rendszeres használatához.



3.13 Ranglista



3.14 Barát Rendszer

3.4.9. Profilbeállítások

A profil menüpontban a felhasználó kezelheti személyes adatait és edzési beállításait. Módosítható például a név, az e-mail-cím, a felhasználónév és a jelszó. Ezen kívül a felhasználó szerkesztheti a kérdőívben megadott válaszait is.

Az edzési profilban módosítható a magasság, a testsúly, a cél és az edzési preferenciák. A rendszer ezek alapján képes különböző számításokat végezni, például BMI, BMR és TDEE értékeket meghatározni.

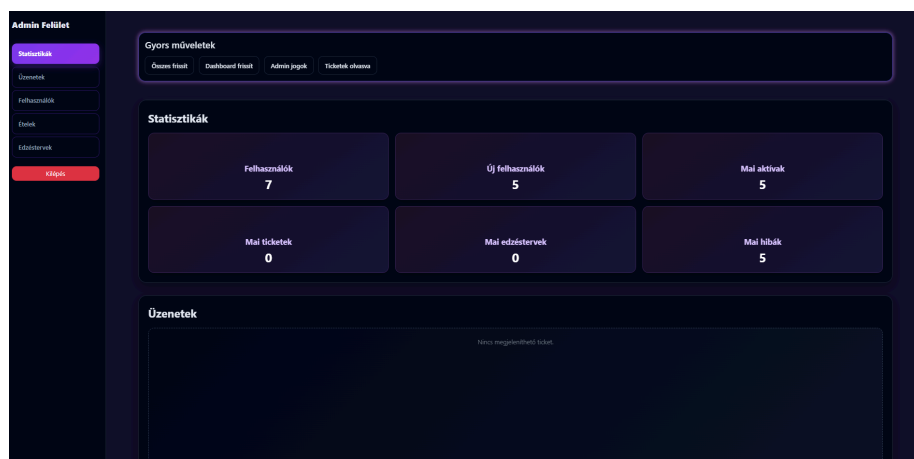
3.15 Profilbeállítások/szerkesztés

3.4.10. Adminisztrációs felület

Az adminisztrációs felület csak admin jogosultsággal rendelkező felhasználók számára érhető el. A felület célja a rendszer karbantartásának és felügyeletének támogatása.

Az adminisztrációs felületen elérhető főbb funkciók:

- felhasználók listázása és kezelése
- fiókok aktiválása vagy deaktiválása
- jogosultságok kezelése (admin)
- receptek és egyéb tartalmak moderálása
- hibajegyek kezelése
- rendszer statisztikák megtekintése
- naplózott események ellenőrzése



3.16 Admin felület

3.5. Technikai információk

3.5.1. Rendszer Architektúra

A MuscleMind kliens-szerver architektúrára épül. A frontend a böngészőben fut, a backend pedig Node.js és Express.js alapokon kezeli a logikát, az adatbázis műveleteket és a felhasználói kéréseket.

A rendszer főbb technológiái:

- Backend: Node.js és Express.js
- Frontend: HTML, CSS, Bootstrap 5, JavaScript
- Adatbázis: MySQL 8.0 vagy újabb
- Munkamenet kezelés: express-session
- Jelszóvédelem: bcrypt
- E-mail-küldés: Brevo API

3.5.2. Biztonsági megoldások

Az alkalmazás több biztonsági megoldást is alkalmaz a felhasználói adatok védelme érdekében. A jelszavak nem olvasható formában, hanem bcrypt segítségével hashelve kerülnek eltárolásra. A bejelentkezett állapot munkamenet alapú hitelesítéssel működik.

A rendszer további védelmi megoldásai:

- SQL injection elleni védelem előkészített lekérdezésekkel
- XSS-támadások elleni védelem bemeneti adatok szűrésével
- CSRF-védelem
- munkamenet automatikus lejárat 30 perc inaktivitás után

3.6. Tervezett fejlesztések

3.6.1. Pontosabb muscle group XP számítás aktivitási szorzókkal

3.6.2. Adaptív edzésterv-ajánló rendszer

3.6.3. Profilkép-feltöltés és recept komment szekció

3.6.4. Gamifikációs és motivációs rendszer bevezetése

- Jelvények
- Heti kihívások
- Teljesítményalapú mérföldkövek

3.6.5. AI személyes tanácsadó chatbot implementálása

4. Csapatmunka megvalósítása

Szalontai Olivér János	Tamás Dávid
Regisztráció	Főoldal
Bejelentkezés	Profil szerkesztés
Felhasználói kérdőív	Barátok/Ranglista
Statisztika	Statisztika
Edzésterv	Naptár
Kapcsolat/Ticket	Izomkövető
Admin felület	Étrend

Ez a táblázat ábrázolja a projekt főbb elemeinek személyenkénti felosztását. Minden megkezdés előtt elemeztük egymás kódjának logikáját illetve elemeztük a lehetséges hibáit. A statisztika oldal olyan indokból szerepel mindkettőnkél mivel Tamás Dávid a statisztika oldalon kiírt adatoknak kiszámítása illetve adatbázisba való betöltése volt a feladata és Szalontai Olivérnek pedig az adatbázisban eltárolt adatok megjelenítése, formázása illetve a grafikonokhoz szükséges adatok betáplálása volt a feladata.

5. Összefoglalás

A MuscleMind esetében a célunk az volt, hogy egy használható, átlátható edzés- és életmód-alkalmazást adjunk át, ami végigvezeti a felhasználót a regisztrációtól a napi szintű tervezésig. Saját megítélésünk szerint a fő szakmai célok nagy részét sikerült elérni: van stabil backend, működő autentikáció, adatokkal feltöltött funkciók (edzésterv, naptár, ranglista, receptek, statisztikák), és ezekhez egy egységesen használható frontend. A projekt egészét most már kész termékként tudjuk nézni, amin persze lenne még mit fejleszteni, de a magja működik és jól bővíthető.

A legnagyobb kihívást a különböző modulok összefésülése jelentette. Az adatmodell kialakítása, a jogosultságkezelés, valamint a frontenden a sok oldal közötti egységes viselkedés és stílus megtartása időigényes volt. Itt tanultuk a legtöbbet arról, hogyan kell a backend és frontend döntéseket összehangolni. A statisztikák és a naptár logikája szintén több próbát igényelt, mire stabilan működött. A statisztikák a számítások miatt leginkább.

Összességében elégedettek vagyunk az elkészült munkával: komplex feladatot sikerült végigvinni, és a rendszer több szempontból is használható. A projekt során sokat fejlődtünk adatbázis-tervezésben, API-k felépítésében és a UI részleteinek kezelésében, ami későbbi munkákhoz is jó alapot ad.

6. Ábrajegyzék

2.1. ábra: A MuscleMind rendszer architektúrája	5
2.2. ábra: statikus front kiszolgálás	5
2.3. ábra: API mount	6
2.4. ábra: regisztráció POST fetch	7
2.5. ábra: GET fetch	8
2.6. ábra: users tábla felépítése & létrehozása	10
2.7. ábra: reset-tokens felépítése & létrehozása	11
2.8. ábra: felhasználói profil tábla felépítése	12
2.9. ábra: felhasználó súly tábla felépítés	13
2.10. ábra: alap edzésterv táblák felépítése	14
2.11. ábra: gyakorlatok tábla felépítése	15
2.12. ábra: naptár törzs tábláinak felépítése	16
2.13. ábra: recept tábla felépítés	19
2.14. ábra: allergének tárolása és összekötés	20
2.15. ábra: kapcsolatfelvétel törzs táblája	21
2.16. ábra: barátrendszer tábla felépítése	22
2.17. ábra: xp-t tároló táblák	26
2.18. ábra: felhasználói statisztika tárolás	27
2.18. ábra: táblák közti kapcsolat példa	30
2.19. ábra: API szerkezetének központi pontja	34
2.20. ábra: regisztráció POST api	36
2.21. ábra: bejelentkezés validáció	38
2.22. ábra: session vizsgálatok	39
2.23. ábra: user metrics elmentése vagy frissítése	41
2.24. ábra: jelszó visszaállítás - új jelszó mentése	43
2.25. ábra: óránként lejárt token törlés	43
2.26. ábra: token_expire_del query	43
2.27. ábra: felhasználó mutatószámokat kiszámító függv.	45
2.28. ábra: új edzésterv létrehozása	47
2.29. ábra: naptár adatainak lekérése	49
2.30. ábra: edzésterv halasztó függv.	51
2.31. ábra: lezárás után statisztika számító függv.	54
2.31. ábra: barát jelölés függv.	56
2.32. ábra: log függvények	60
3.1. ábra: Regisztráció	84
3.2. ábra: Bejelentkezés	85
3.3. ábra: Főoldal	87
3.4. ábra: Edzés emlékeztető	87
3.5. ábra: Edzésterv létrehozás, ajánlott edzéstervek, szűrés	89
3.6. ábra: Étrend Kezelő oldal felépítése, szűrő	90
3.7. ábra: Étrend Kezelő oldal új recept létrehozása	90
3.8. ábra: Étrend Kezelő oldal közösségi receptek, saját receptek	91
3.9. ábra: Naptár oldal edzésekkel feltöltve	92

3.10. ábra: Edzés kezelés, sorozatok, ismétlés, súly hozzáadása	92
3.11. ábra: Statisztika oldal (1)	93
3.12. ábra: Statisztika oldal (2.) Grafikonos felület	94
3.13. ábra: Ranglista	95
3.14. ábra: Barát Rendszer	95
3.15. ábra: Profilbeállítások/szerkesztés	96
3.16. ábra: Admin felület	97

7. Irodalomjegyzék

- Node.js dokumentáció (ESM, futtatás): <https://nodejs.org/en/docs> 04.12
- Express.js útmutató (routing): <https://expressjs.com/en/guide/routing.html> 04.13
- Express.js middleware alapok: <https://expressjs.com/en/guide/using-middleware.html> 04.13
- express-session docs: <https://www.npmjs.com/package/express-session> 04.14
- MDN Fetch API: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API 04.15
- Bootstrap 5 layout (grid): <https://getbootstrap.com/docs/5.3/layout/grid/> 02.03
- Bootstrap 5 components (modal):
<https://getbootstrap.com/docs/5.3/components/modal/> 02.13
- Chart.js getting started: <https://www.chartjs.org/docs/latest/getting-started/> 04.22
- Chart.js configuration: <https://www.chartjs.org/docs/latest/configuration/> 04.22
- Brevo API (SMTP email):
<https://developers.brevo.com/docs/send-a-transactional-email> 04.16
- Express.js hibakezelés: <https://expressjs.com/en/guide/error-handling.html> 04.12
- Express.js statikus fájlok kiszolgálása: <https://expressjs.com/en/starter/static-files.html> 04.17
- MySQL JOIN szintaxis: <https://dev.mysql.com/doc/refman/8.0/en/join.html> 02.28
- MDN FormData: <https://developer.mozilla.org/en-US/docs/Web/API/FormData> 04.03
- Bootstrap Offcanvas komponens:
<https://getbootstrap.com/docs/5.3/components/offcanvas/> 02.05
- Bootstrap Forms (form-control, select):
<https://getbootstrap.com/docs/5.3/forms/overview/> 02.12
- PNG SVG converter:
<https://www.adobe.com/express/feature/image/convert/png-to-svg> 04.28
- <https://chatgpt.com/> 05.04.

1. Bevezetés.....	1
1.1 Köszönetnyilvánítás.....	1
1.2 Témaválasztás.....	1
1.3 Használt technológiák.....	2
2. Fejlesztői dokumentáció.....	4
2.1. Projekt áttekintése.....	4
2.2. Architektúra.....	5
2.2.1 Backend.....	5
2.2.2. Frontend.....	6
2.2.3. Adatbázis.....	8
2.2.3.1. Adatbázis-terv.....	8
2.2.3.2. Kulcsfontosságú entitások.....	9
2.2.3.2.1. Felhasználó és jelszó-visszaállítás.....	9
2.2.3.2.2. Profil és mért adatok.....	11
2.2.3.2.3. Edzéstervek és naptár.....	13
2.2.3.2.4. Recept- és jegyrendszer.....	17
2.2.3.2.5. Ranglista és barátok.....	21
2.2.3.2.6. Statisztikai adatok.....	23
2.2.3.3. Adatmodell kiegészítések.....	26
2.3. Program komponensek.....	29
2.3.1. Futtatási környezet.....	30
2.3.2. server.js.....	30
2.3.3. API routing.....	33
2.3.4. Middleware.....	35
2.3.5. Adatbázis szolgáltatások.....	38
2.4. Algoritmusok.....	41
2.4.1. Auth, jelszó és tokenkezelés.....	41
2.4.2. Kérdőív és metrika számítás.....	43
2.4.3. Edzésterv generálás és naptár.....	45
2.4.3.1. Új felhasználói edzésterv létrehozása.....	45
2.4.3.2. Naptárgenerálás.....	46
2.4.3.3. Naptár-átütemezés.....	48
2.4.4. Edzés lezárása és statisztikák.....	50
2.4.5. Ranglista és barátok.....	54
2.4.6. Receptek.....	56
2.5. Tesztelés.....	59
2.5.1. Tesztelési környezet.....	59
2.5.2 Tesztfájlok áttekintése.....	60
2.5.3. Backend tesztek.....	62
2.5.3.1. backend/tests/basic.test.js.....	62
2.5.3.2. backend/tests/api404.test.js.....	62
2.5.3.3. backend/tests/authMiddleware.test.js.....	63
2.5.3.4. backend/tests/adminMiddleware.test.js.....	64
2.5.3.5. backend/tests/registrationMiddleware.test.js.....	64

2.5.3.6. backend/tests/ticketMiddleware.test.js.....	65
2.5.3.7. backend/tests/workoutValidation.test.js.....	66
2.5.4. Frontend tesztek.....	66
2.5.4.1. frontend/tests/api.test.js.....	66
2.5.4.2. frontend/tests/login.test.js.....	67
2.5.4.3. frontend/tests/register.test.js.....	68
2.5.5. Pozitív és negatív tesztesetek aránya.....	69
2.5.6. Teszteredmények.....	69
2.5.7. Coverage eredmények.....	70
2.5.8. Összegzés.....	71
2.6. Továbbfejlesztési lehetőségek.....	71
2.6.1. Pontosabb muscle group XP számítás aktivitási szorzókkal.....	71
2.6.2. Adaptív edzésterv-ajánló rendszer.....	72
2.6.3. Profilkép-feltöltés és recept komment szekció.....	72
2.6.4. Gamifikációs és motivációs rendszer bevezetése.....	72
2.6.5. AI személyes tanácsadó chatbot implementálása.....	73
2.7. Összegzés.....	73
3. Felhasználói dokumentáció.....	74
3.1. A program célja és főbb funkciói.....	74
3.1.1. A program célja.....	74
3.1.2. Főbb funkciók.....	74
3.2. Szükséges hardver- és szoftverkörnyezet.....	77
3.2.1. Hardverkövetelmények.....	77
3.2.2. Szoftver Követelmények.....	77
3.3. Telepítés és indítás.....	79
3.3.1. Előfeltételek.....	79
3.3.2. A projekt letöltése.....	79
3.3.3. Backend telepítése.....	79
3.3.4. Adatbázis beállítása.....	81
3.3.5. Környezeti változók beállítása.....	81
3.3.6. A szerver indítása.....	81
3.3.7. Az alkalmazás elérése.....	82
3.3.8. Lehetséges telepítési problémák.....	82
3.4. A program részletes bemutatása.....	83
3.4.1. Regisztráció és bejelentkezés.....	83
3.4.2. Kérdőív kitöltése.....	85
3.4.3. Főoldal és irányítópult.....	86
3.4.4. Edzéstervek kezelése.....	88
3.4.5. Étrend és receptek kezelése.....	89
3.4.6. Naptár és edzésnapló.....	91
3.4.7. Statisztikák.....	93
3.4.8. Ranglista és közösségi funkciók.....	94
3.4.9. Profilbeállítások.....	95
3.4.10. Adminisztrációs felület.....	96

3.5. Technikai információk.....	97
3.5.1. Rendszer Architektúra.....	97
3.5.2. Biztonsági megoldások.....	97
3.6. Tervezett fejlesztések.....	98
3.6.1. Pontosabb muscle group XP számítás aktivitási szorzókkal.....	98
3.6.2. Adaptív edzésterv-ajánló rendszer.....	98
3.6.3. Profilkép-feltöltés és recept komment szekció.....	98
3.6.4. Gamifikációs és motivációs rendszer bevezetése.....	98
3.6.5. AI személyes tanácsadó chatbot implementálása.....	98
4. Csapatmunka megvalósítása.....	99
5. Összefoglalás.....	100
6. Ábrajegyzék.....	101
7. Irodalomjegyzék.....	102